



M256 Unit 11

UNDERGRADUATE COMPUTING

Software development with Java



Human-computer
interaction

Unit **11**

This publication forms part of an Open University course M256 *Software development with Java*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2007.

Copyright © 2007 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 1559 0

CONTENTS

1	Introduction	5
1.1	What is human–computer interaction?	5
1.2	The aims of this unit	6
1.3	Studying this unit	7
2	Why is human–computer interaction important?	8
2.1	Safety	8
2.2	Market share	9
2.3	Enabling the otherwise impossible or impractical	12
2.4	Training	13
2.5	Extraordinary human–computer interaction	14
2.6	Productivity	15
3	Human–computer interaction fundamentals	16
3.1	Socio-technical systems and user interfaces	16
3.2	User interface styles	17
3.3	Designing for users	19
4	Introduction to user interface design	21
4.1	Design principles	21
4.2	Reflecting on the user	25
4.3	User interface metaphors	28
5	Widget functionality	36
5.1	Menu bars and menus	36
5.2	Buttons	38
5.3	Radio buttons and check boxes	39
5.4	Windows and dialogue boxes	41
5.5	Other widgets	44
6	Summary	48
	Glossary	49
	References	52
	Acknowledgements	52
	Index	53

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Composer

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

A principal aim of M256 is to give you a clear understanding of key aspects of object-oriented software development. In accordance with accepted software engineering practice we have considered the user interface and the rest of the system (that is, the core system) as separate – though, of course, interacting – entities. This means that different teams of people might work in parallel on the core system and the user interface, with the development of each informing the other.

Because our example systems are relatively simple, we have been able until now to focus solely on developing the core system. You might think of this as the first of several iterations of an iterative development process. Within this first iteration, the time has come to consider the user interface explicitly. To do that properly, you need to understand something of the history and current significance of **human–computer interaction (HCI)** – broadly, the study of how computers and people work together. That is the purpose of this unit and the next.

HCI is rather different from other parts of computing, because of its emphasis on people, as well as on software. Because it is cross-disciplinary, HCI can be complex when looked upon as a formal science. However, we shall focus on HCI as the study of *how to make software work better with people*.

Considered from this practical perspective most of the basics of HCI are relatively simple. Both in this unit and in *Unit 12* we will bring out this simplicity by encouraging you to see the connections between issues in HCI and your own everyday experience. Perhaps unexpectedly, you will also discover along the way that the historical origins of HCI are closely bound up with the beginnings of object-oriented ideas.

You will find that, given the nature of the subject matter, *Units 11* and *12* are more discursive than the preceding units.

1.1 What is human–computer interaction?

The term HCI refers to at least two different activities that are often confused. It is important to be aware of the difference.

Human–computer interaction as a science

First of all, HCI is used to refer to the *scientific* study of the interaction between humans and computers. This area of research necessarily combines aspects of both computer science and social science. In particular, HCI draws on and adapts the methods of social sciences such as psychology and ethnography. The primary aim of HCI in this sense is to discover, test and refine reliable knowledge about how people and computers interact. Though this activity is very useful and relevant to M256, it will not be the primary HCI activity that we will focus on in the course.

Loosely speaking, ethnography is a scientific discipline studying what people do in realistic situations, as opposed to what they do under artificial laboratory conditions.

Human–computer interaction as a practical and professional activity

A second meaning of HCI is the *design of interactions* between humans and computers. This activity has exactly the same focus on interactions between humans and computers, but is concerned less with contributions to knowledge as such, and more

with the practical business of striving for good interfaces and interactions (applying relevant scientific knowledge). This is the sense of HCI that will be the focus of this course.

Some people make a living doing practical HCI: designing user interfaces for new products, or improving existing user interfaces. But much practical HCI is carried out by computing generalists. Practical HCI is a craft that computing professionals should be able to turn their hand to when necessary.

The scope of human–computer interaction

The scope of HCI is not limited to ‘traditional’ computer systems. HCI covers what are known as **interactive products**, which include not only conventional desktop systems, but also any computer-based devices that require users to engage with them.

Examples include telephone voice menu systems, mobile telephones and personal digital assistants (palmtop computers). HCI in this course will largely be confined to the context of traditional computer systems, though we will at times consider other interactive products as a source of inspiration for HCI ideas.

Practical HCI involves consideration of a wide variety of issues, of which the following constitute a small sample.

- ▶ Input and output devices. For example, if users have visual impairments, the technology must meet their particular needs.
- ▶ The cognitive characteristics of the users. How much information can they remember?
- ▶ The background and knowledge of the users. For example, are they young children or business users?
- ▶ The tasks the users wish to carry out. For example, are they controlling a nuclear power station or writing letters?
- ▶ The types of interface available. Most modern interfaces are **graphical user interfaces (GUIs or GUI interfaces)**, combining visual elements such as buttons and lists. However there are alternative kinds such as command-line interfaces and menu interfaces.

‘GUI interfaces’ is syntactically incorrect if GUI is expressed in words, but it is widely used among software developers.

1.2 The aims of this unit

It would be impossible to cover all the issues listed above in detail, so a selection of topics has been chosen for this unit and for *Unit 12*, which underpin a single message: that user interface design should be centred on the needs and characteristics of the user. Consequently, the aims of this unit are:

- ▶ to describe the importance of HCI (Section 2);
- ▶ to outline some major historical developments in user interface styles, some of which will be further analysed in *Unit 12* (Section 3);
- ▶ to introduce and analyse some of the basic concepts of HCI including metaphor and direct manipulation, and the idea of design principles – again these will be pursued in *Unit 12* (Section 4);
- ▶ to analyse some simple elements of common GUI interfaces, both in terms of functionality and in terms of relationship to design ideas (Section 5).

In *Unit 12*, building on the insights and skills gained from this unit, you will explore the connections between object-oriented ideas and graphical user interfaces. You will also learn effective and widely applicable techniques for designing user interfaces from scratch and for improving the design of user interfaces.

1.3 Studying this unit

The unit contains a number of SAQs and paper-based exercises, which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on.

As this unit is more reflective and discursive in nature than the preceding units, some of the exercises and SAQs are ‘thought experiments’, and as a consequence they are open-ended. There is no practical work specifically associated with this unit. However, when studying it, you may find it helpful to refer to the computer as a source of ideas and illustrations. All of the work in the course that you have carried out using a computer, and indeed anything from your experience as a computer user, will be a useful source of experiences for reflecting on in this unit.

The concepts in this unit are illustrated with a number of examples, some of which are given in boxed format. Remember that this means you are not expected to commit to memory the details of the example, though you should be able to identify the example as an illustration of a particular concept.

We expect that Sections 4 and 5 will require the most study time of the sections in this unit.

2

Why is human–computer interaction important?

One pragmatic reason why HCI is important is that the proportion of code currently being written that is solely concerned with user interfaces is commonly estimated to be around 70%. It follows that studying interface code and finding ways to make it easier to write and more reliable are concerns of great importance in producing software.

Another reason why HCI is of importance is that the everyday activities of an increasing number of people involve using interactive products such as mobile telephones, MP3 players, navigation devices, ‘smart’ home fittings, the Web, computer-based tills and ATMs, ticket machines and ecommerce systems of all kinds. Consequently, the **usability** of an interactive product is an important idea. Definitions of the term vary, but typically it addresses the following questions.

- ▶ How easily and quickly can users achieve their goals?
- ▶ How quickly can users learn to use the product’s interface?
- ▶ What is the attitude of the users to the interface? Do they like it?

In this section you will learn how software can be useless, less useful than it should be, overly time-consuming to learn or employ, or even downright dangerous if it has poor usability. Often users blame themselves, but the problem very frequently lies in poor user interface design. You will also learn about some products whose usability has made them a great success.

ATM is the acronym for ‘automated teller machine’ – cashpoint machines that enable users to withdraw cash from bank/building society accounts.

Unit 12 looks in more detail at the idea of usability.

2.1 Safety

Safety is particularly crucial in certain systems – sometimes called **safety-critical systems** – such as those found in ambulance dispatch, aerospace, medicine, and the nuclear and chemical industries.

Of 1000 computer-related deaths examined in a government-funded study carried out in the 1990s, some 4% were due to faulty software, some 3% related to problems with hardware and electromagnetic radiation, while over 90% were attributed to human factors and problems in human–computer interaction. Indications are that little has changed since then. In the examples in the box below, a user interface was a direct factor in the loss of life or a costly disaster.

Safety disasters

Three Mile Island

In a notorious nuclear-power disaster in 1979, the nuclear reactor at Three Mile Island in the United States came very close to a meltdown. Much attention was focused on the poor user interface aspects of the control panel. In particular:

- ▶ A light indicated that a valve had been closed when in fact it was open.
- ▶ A caution tag attached to a controller obscured an indicator light.
- ▶ There were some 1500 alarms that individually facilitated control in normal day-to-day situations. Many of these went off at once, spreading panic and hindering communication. A single button would have switched off all the alarms, but was not used because the information about individual alarms would have been lost.

Indian Airlines Flight 605

In the 1990s Indian Airlines Flight 605 (an Airbus 320) crashed, killing 98 people. *Flight International* magazine blamed poor understanding by the pilot of the aircraft control interface.

Iran Air Flight 655

In 1988 the United States Navy accidentally shot down an unarmed civilian jetliner, Iran Air Flight 655. The onboard naval computer provided entirely accurate information; however, crucial information about position, heading and altitude was not summarised on a single screen, but displayed on different consoles, making correlation difficult. Although the data clearly indicated that the aircraft was a civilian flight, misinterpretation by the operator led to its being identified as a military flight and it was shot down.

2.2 Market share

A key determinant of the success of systems in the marketplace is the quality of the user interface.

Google

In the mid-1990s, when the Web was in its infancy, there were many competing search engines such as Yahoo!, AltaVista and Lycos. Google was a relative latecomer. It had particularly good technical features, but initially these were not always obvious to users.

However, Google did immediately stand out from its competitors for other reasons. For example, while the AltaVista search page was covered with advertisements, so that it could be hard to find the search box quickly (see Figure 1), Google presented a clean, minimalist web page, with nothing to distract the user from the core task of searching (see Figure 2).

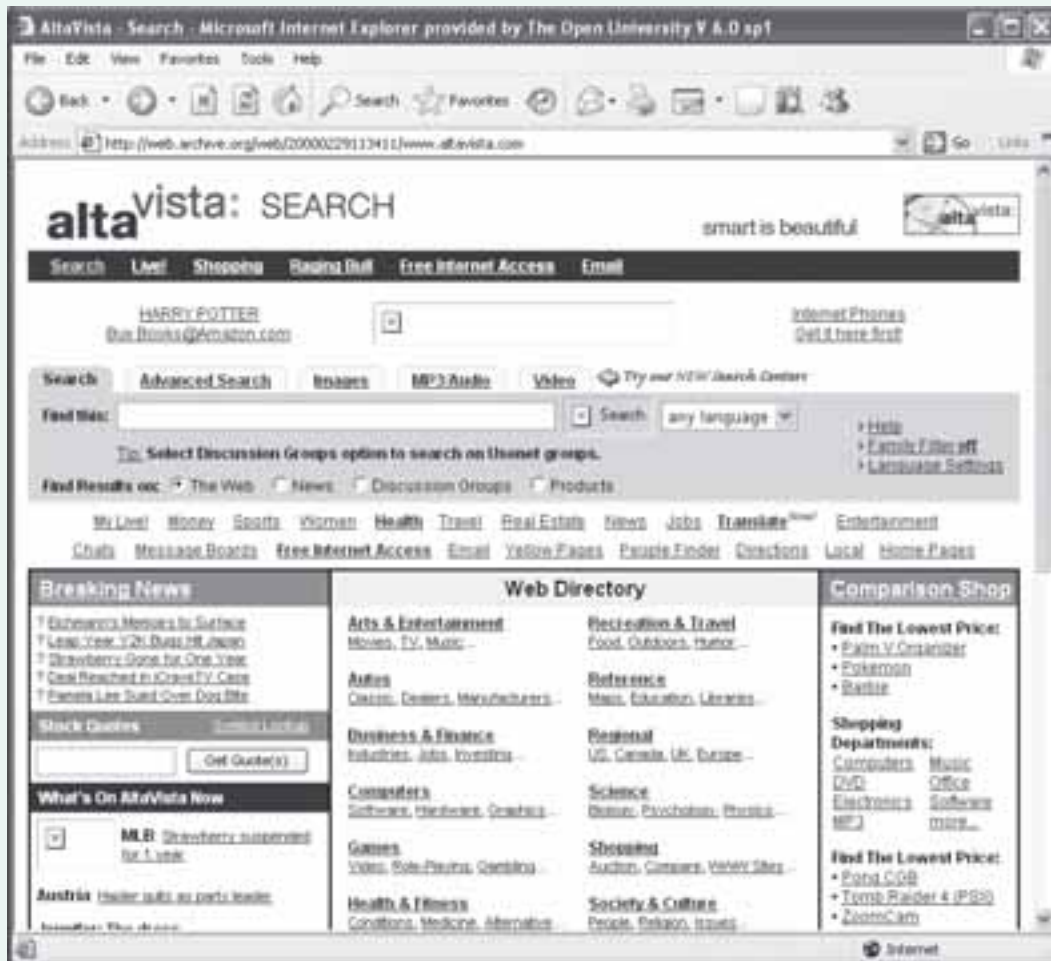


Figure 1 The AltaVista search page at the time when Google arrived



Figure 2 A typical Google search page

Not only was the Google user interface carefully designed to be fit for purpose, but also the Google business ethos gave centre stage to HCI. A key business issue of the time was to find a way to generate advertising revenues without alienating users with intrusive advertisements. Google decided that no advertisements should be displayed until a search was carried out, and that the advertisements should then be based on the content of the user's search and displayed discreetly as in the example in Figure 3. This crucial decision combined careful attention to both business and HCI issues.



Figure 3 Results of a Google search

Google went on to dominate the market on the basis of various strengths, but its well-designed user interface played a significant part in helping it to obtain initial attention and market share. This connection between HCI issues and business success is far more common than is generally realised.

iPod

When MP3 players and commercial downloadable music first became widely available in the early 2000s, Apple rapidly moved to a dominant position in the market with the iPod MP3 player and the accompanying iTunes software. While it is difficult to attribute this success, with certainty, to particular design decisions, three contributing factors are widely considered to have been centrally important:

- 1 The iPod was designed to be an aesthetically pleasing physical object. Aesthetics has been gaining importance as an issue in HCI (though it is generally considered a mistake to pursue good looks at the expense of usability).
- 2 The user interface of the iPod was designed with great attention to usability: the controls were relatively simple to learn and use and the interface gave clear feedback at all times.
- 3 Since an iPod generally needed to be connected periodically to an internet-enabled computer, the integration of the user interfaces of the iPod and the host computer was an important issue. Integration was generally considered to be handled better by the iPod than by its competitors.

These three issues, all HCI-related, helped the iPod quickly dominate the MP3 player market.

Countless other everyday examples of the impact of user interfaces on market share are provided by ecommerce websites. When there are a variety of competing sites for a given product or service, if users find a particular site to be hard to search, hard to understand or slow to respond, they will often go elsewhere.

2.3 Enabling the otherwise impossible or impractical

Good HCI does not merely make software safer to use or more widely sold. Sometimes, changes in user interfaces are so enormous that they make possible entirely new tasks, or make feasible tasks that were formerly impractical.

The Web

The Web is a spectacular example of enabling seemingly impractical tasks. The internet only gained mass following when the Web provided access to it with a consistent, easy-to-use interface. Similar communications facilities, but without the convenient user interfaces, had existed for at least 20 years, but their lack of usability rendered use of the internet impractical beyond tiny, technically skilled communities. Of course, many other factors were involved in acceptance of the Web (such as the simplicity of the network protocols involved, and the ease of authoring), but the HCI factors were of key importance.

Virtual reality

An excellent example of making new kinds of task possible is seen in **virtual reality systems**. *Virtual reality* refers to ways of interacting with a computer (or indeed interacting with other people or places via a computer), which may vary greatly but have the following factors in common.

- ▶ A sense of direct physical presence. The technology creates three-dimensional sensory cues that may be visual, aural or haptic (concerned with a sense of touch or force on the body), to give a subjective sense of direct experience or physical presence.
- ▶ Natural interaction. Computer-generated 'objects' may be manipulated using much the same gestures one would use to manipulate their real counterparts, e.g. picking up, turning around and throwing.

The haptic virtual reality system GROPE, described below, was developed in the 1960s and 1970s. It lets chemists explore, using sight, sound and touch, how organic molecules interact. This user interface does not just allow something to be done faster or more easily – it enables experiences that would otherwise be impossible.

GROPE: a force-feedback interface

GROPE is a touch-based interface which allows scientists to feel simulated electrical forces involved in trying to fit molecules into three-dimensional structures. Finding out about such things can be very important in the design of new drugs, and in answering questions in molecular biology. The system uses a remote-manipulator force-feedback arm (see Figure 4). By grasping the handpiece, which is suspended from the ceiling, and by using shoulder, wrist and finger muscles to move it, a user can manipulate computer-generated virtual objects, such as models of molecules, which can offer resistance.

In a typical interaction, the user stands in front of a large computer screen, holding the arm. In Figure 4, there are two molecules visible on the screen; the one in the foreground twists and moves with the arm. Attempts to ‘dock’ (join together) the molecules may lead to repulsions and resistances, until the correct angle of approach is found. Audible clicks are used when virtual contacts are made. In later versions of the system, computer-driven headsets were worn to give a three-dimensional view of the molecules. Studies found that this system improved chemists’ understanding of the details of the receptor site and its force fields, and of why particular combinations of molecules dock well or poorly.



Figure 4 The Grope III system

The success of GROPE stimulated development of what is today a vast range of virtual reality applications. For example, virtual reality simulators are now in common use in medical schools. Such simulators give students the chance to work their way through layers of tissues and muscles sequentially, to inspect organs individually and to ‘see’ anatomy from unusual viewpoints (e.g. from the inside of a body). GROPE is, however, still widely respected not least because it truly enables the otherwise impossible – there is no real-world equivalent to ‘feeling’ molecules, as there is to virtual surgery.

2.4 Training

In the infancy of computing, computers and software were expensive, and the salaries of the people who used them were small in comparison. Now the reverse is generally true. However, when considering which software package or computer to buy, businesses sometimes make price comparisons without considering the costs of training the users.

But the usability of an interface can have a big effect on training costs. For example, when the Apple Macintosh computer first came out, it was noticed that, in contrast to competing systems of the time, once a user had learnt to use one application on the Macintosh, it took much less time to learn to use any other application. This was mainly due to the fact that the Macintosh, unlike its competitors, enforced consistency of operation onto interfaces – much of what held in the interface of one application held in the interface of any other application. This is now increasingly true of Microsoft Windows and other operating systems.

You will learn more about the importance of consistency in user interfaces in *Unit 12*.

2.5 Extraordinary human–computer interaction

Extraordinary HCI is a term used to describe human–computer interaction in situations where the user has special needs. For people with physical or sensory disabilities, computers can be hugely empowering devices. In many cases, this power depends on the user interface. An interesting example is provided by the famous physicist, Stephen Hawking, whose mobility is very limited, and who cannot speak. Hawking's computer system, in one well-analysed version – the Equaliser, detailed below – uses many techniques to speed up input via a single switch.

The Equaliser system

The Equaliser is an example of a system that is technically relatively simple but is closely tailored to the needs of its user. This system employs specialised hardware and software to enable the user to control a speech synthesiser and a word processor using just a single switch.

The Equaliser has evolved through many versions. The method of input in one well-known version uses a series of menus arranged on a screen, as shown in Figure 5. Different parts of the screen are automatically highlighted in turn by the system in a rapid and regular repeating sweep. The user selects the portion of the screen that is currently of interest by clicking the switch when that area is highlighted.

Many ingenious strategies are used to minimise the number of clicks required for common speaking and writing tasks. For example, in one area of the top-level screen, the 36 most commonly used words in the English language are arranged according to frequency of use. Another part of the top-level screen allows less common words to be spelt out letter by letter, and another gives access to a dictionary list from which complete words may be chosen. When a complete word has been spelt out or chosen by whatever means, the system displays a row of up to six predictions about what the next word is likely to be. These guesses may be ignored, or one of them may be chosen.

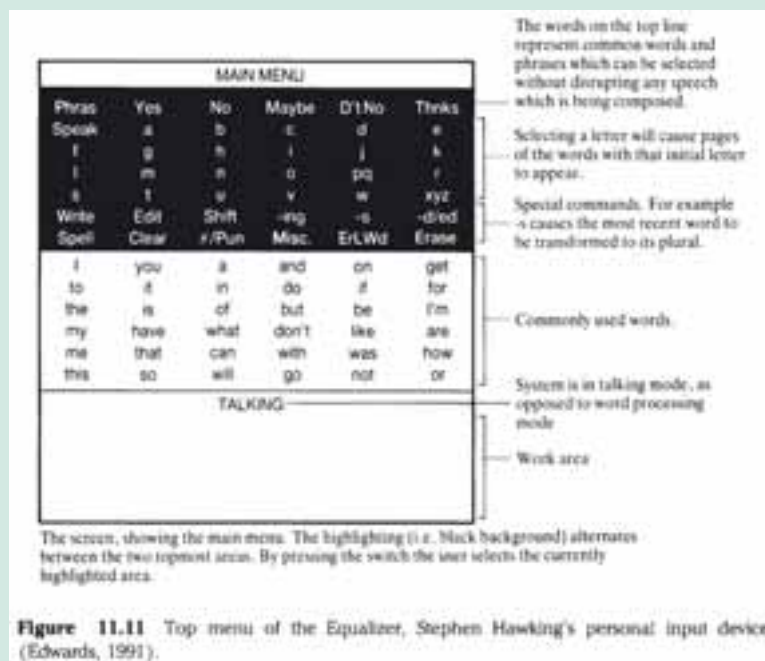


Figure 5 The interface of one well-known version of the Equaliser

If you are familiar with predictive text systems for inputting mobile telephone text messages, parts of the above description may sound familiar to you. This is one example where a technique explored extensively in extraordinary HCI went on to influence mass-market user interfaces.

2.6 Productivity

Decreasing the time taken to carry out a frequently performed commercial transaction can allow more transactions to be fitted into a given space of time, while reducing the demands on those carrying out the transaction. It follows that good user interface design can save a company very large amounts of money.

To take a simple example, when contacting companies by phone, instead of asking for your address many of them now just ask for your postcode. The operator can then retrieve most of your address on screen in a second or two, from a database of postcodes, and need only verify this with you, and then ask for, say, your house number. Note that this example emphasises that HCI covers more than just the layout of the screen: it also encompasses what facilities and sources of information are made available to users (here operators).

SAQ 1

Give three reasons why HCI is important.

ANSWER.....

Reasons can be drawn from any of the following:

- ▶ increasing safety;
- ▶ increasing market share;
- ▶ enabling the otherwise impossible or impractical;
- ▶ facilitating training;
- ▶ facilitating extraordinary HCI;
- ▶ increasing productivity.

SAQ 2

Above, we noted that consistency of the early Apple Macintosh interfaces enabled users to learn quickly how to use the computer; that is, the consistency *facilitated training*.

How might a consistent interface contribute to the following.

- (a) Safety?
- (b) Productivity?

ANSWER.....

- (a) With a consistent interface, users are less likely to make errors, thus contributing to safety.
- (b) With a consistent interface, users are likely to work more quickly and accurately, thus contributing to productivity.

In this section we have illustrated how important the usability of user interfaces is to everyday life. In Section 3 you will learn about some of the fundamental ideas that are relevant to HCI, and in particular to the creation of usable interfaces.

3

Human–computer interaction fundamentals

Now that you are – we hope – convinced of the importance of HCI, this section:

- ▶ provides definitions of some key concepts;
- ▶ introduces some important HCI issues that are covered in more depth later in this unit and the next.

HCI as an area of study is a relatively young discipline, and consequently the meaning of some terms is not precisely defined or universally agreed. However, the definitions used here are at least good ‘working’ definitions, which will help you understand essential HCI ideas.

3.1

Socio-technical systems and user interfaces

HCI incorporates all of the factors that affect how people and computer systems work together; hence it involves not only hardware, software and people, but also supporting elements such as documentation and training. Given this broad remit of HCI, it makes sense to consider how we think about systems and their boundaries.

Computer systems involve a combination of hardware and software. However, in most situations, tasks are carried out through the operation of a computer system by one or more people. Consequently, the most useful ‘system’ to focus on is often the wider system comprising the hardware, software and users. Any system comprising both human and technical elements is said to be a **socio-technical system**. Such a system may comprise a single user and a single machine, or thousands of people and scattered networks of computers. In either case, the interactions between the human and technical elements of the system will take place at the *user interface*.

In this unit so far, and indeed in the course so far, we have used the term *user interface* without defining it, because the vast majority of people have a good intuitive understanding of what it means. However, for the purposes of this course the **user interface** of a system may be broadly defined as the features of the system the user can use to communicate with or control the system, and those which the system can use to communicate with or influence the user. There are three aspects to this definition that are worth drawing out explicitly:

- 1 hardware;
- 2 visual elements;
- 3 interaction design.

User interface: hardware

First, there are the purely hardware elements that form part of a user interface. For example, all of the following and more may be part of a user interface: a mouse, a keyboard, a screen, a microphone and speakers. In some systems there might be more exotic user interface hardware, such as motion sensors. In some respects the hardware aspect is the least interesting aspect, since the developer may have little control over what hardware is present.

User interface: visual elements

The second aspect of a user interface is everything that appears on a screen. This concerns the individual visual **widgets** – elements such as buttons, lists etc. on a GUI interface – and is a more interesting aspect for developers, as they typically have control over this part.

Modern computers with good graphical facilities can use widgets in different ways to display countless representations of a system to the user. To most users, the interface *is* the system. The challenge is to design a user interface that enables appropriate interactions between the user and the system and this leads to the third aspect of user interfaces.

Of course, the elements of an interface may not actually be visible: they may be aural or haptic (touch/feel). However, we will concentrate on visual aspects.

User interface: interaction design

The third aspect focuses on the interactions between a computer system and a user, covering issues such as:

- ▶ how tasks are divided up between the system and the user;
- ▶ the extent to which the system or the user controls what happens next;
- ▶ the timing or ordering of dialogue during interactions.

The term **interaction design** is relevant here. You can think of this term in the context of a particular system as referring to the design of any aspects of the system that affect the quality of a user's interaction. This emphasises the philosophy that interactions between people and systems should always be explicitly *designed*, with users' needs at centre stage.

SAQ 3

Describe briefly how, to an interface designer, the second and third aspects of user interfaces are related.

ANSWER.....

Visual elements (second aspect) are selected by the designer and combined to realise the interaction design (third aspect) that they have developed.

3.2 User interface styles

In this subsection we present a very brief history of user interfaces, stressing the different styles through which user interfaces have developed. In *Unit 12*, we will expand on the historical development of graphical user interfaces, concentrating on the connection with the development of object-oriented ideas.

In the early days of computing, the balance of the various elements in socio-technical systems was very different from that today. Computers were large, rare and expensive. The majority of users were specialists working for the military or in scientific laboratories of big corporations, and were able to devote considerable time and effort to learning how to control the computer. They worked on large mainframe computers, and wrote programs in languages that only they understood. Such users had input and output devices such as teletypewriters, which to the present day user seem slow, noisy and inflexible.

Since the late 1970s this has all changed. The introduction of personal computers transformed the way in which non-specialists perceived the computer. For the first time, computers became available to ordinary people in their offices and homes – one

Teletypewriters operated somewhat like a keyboard, with a limited character set. They often contained a device for reading punched paper tape.

consequence of this was that the computer user became a consumer: consumers wanted to be able to purchase a computer system which matched their needs with minimum effort on their part.

Command-line user interfaces

As the market developed, the IBM PC became a widely copied standard. This resulted in a rapid reduction in prices and increasing market domination. When the IBM PC was first released, it provided an operating system called PC-DOS (Personal Computer Disk Operating System). On compatible personal computers, produced by other computer manufacturers, the operating system was called MS-DOS (Microsoft Disk Operating System).

The user interfaces of these operating systems were examples of **command-line interfaces**, which required the user to know the name of the command, the way it was used and the names of the files (or other items of interest) to be manipulated. If the command was typed with even the smallest error it would not work at all, and usually little indication was provided of how to correct it. For example, in order to delete a file named TEST.DAT from a removable disk, the user would have to enter a command such as DEL A:TEST.DAT in response to the prompt C:\> shown in Figure 6.

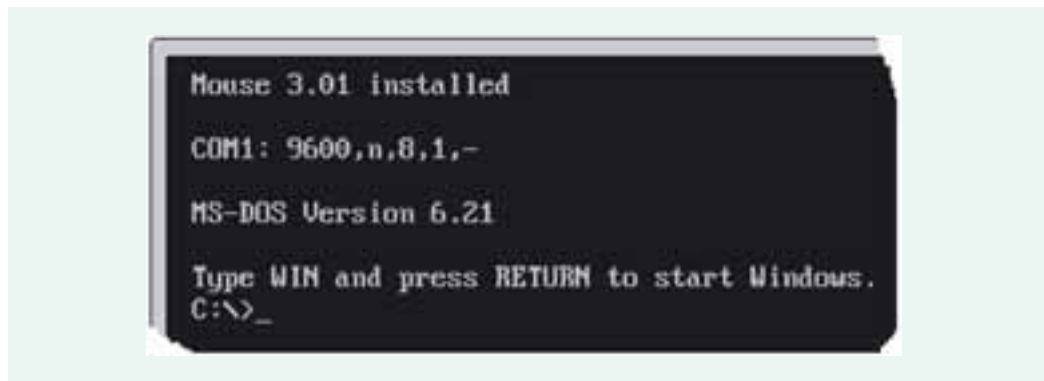


Figure 6 An MS-DOS prompt

Graphical user interfaces

In the early 1970s the computer scientist Alan Kay set up a research group at Xerox PARC (Palo Alto Research Centre) in California. This group created Smalltalk, the pioneering object-oriented language, some of whose ideas were to be borrowed much later by Java. As part of this work they also developed many of the features of modern GUI interfaces. Thus object-oriented technology and GUI interfaces have common roots.

A GUI typically involves widgets such as windows, buttons and menus: graphical elements which can be manipulated by the user to interact with the system. One of the most significant theoretical aspects of this development is that GUI interfaces can have the same functionality as command-line interfaces. In other words, a user can carry out the same tasks but in a different way. For example, to delete a file using a Windows GUI interface, the icon for the file can simply be dragged into the Recycle Bin, giving the same functionality as the MS-DOS delete command.

Inspired by the work of Kay, Apple Computers borrowed and further developed the GUI ideas from the team at Xerox PARC, leading to the Apple Macintosh. For people who were not computer experts, this interface was far more usable than command-line interfaces. It took some time for this approach to become recognised and accepted, but after a number of years, Microsoft began to catch up, leading to the modern GUI interfaces of Microsoft Windows.

An **icon** is a small representational picture used, for example, to label a button.

There are a number of GUI interfaces in current use, though there is a high degree of visual and functional consistency between them. This means that moving between them is relatively straightforward – experience of one helps the user operate the others.

Menu interfaces

In situations where display or processing resources are very limited, GUI interfaces are not always appropriate. Provided the task is simple enough, one simple solution is a **menu interface**.

In a typical menu interface the screen displays a number of options, and the user is asked to select an option, generally by pressing a button or typing in a number. Typically the system will present a sequence of choices of this kind until the task is complete. This kind of user interface is used for such diverse purposes as ATMs, digital TV sets and global positioning systems.

3.3 Designing for users

To create a successful socio-technical system, it is not only the technology that needs to be understood, but also the human element and the way the two interact.

The term **user-centred design** refers to a design culture or design process where users' needs, wants, expectations and limitations are given serious, detailed attention at all stages. In particular, in user-centred design, the user's needs should take priority over engineering or programming convenience, and over designers' or managers' personal preferences.

The behaviour of users can be very varied and unexpected. For example, the Microsoft Usability Laboratory found that when the mouse was first introduced, some users ran the mouse up and down the leg of the table (presumably taking a cue from the word 'mouse'). Other users waved it in the air. It is easy to laugh at such examples and ridicule early users. However, the cause of such miscommunication is often poorly designed user interfaces, which do not take into account the characteristics and background of users. These examples are trivial but, as you saw in Section 2, serious accidents have been caused by interfaces with poor usability.

In fact, the Nintendo Wii controller works precisely by being waved in the air!

An understanding of a variety of academic disciplines is necessary to design interfaces that take these issues into account. Some are given below.

- ▶ *Cognitive psychology* This provides theories about the way individuals think, remember, and tackle problems. For example, the use of colour in an interface should be guided by an understanding of the way in which users typically interpret different colours: red for danger, for example.
- ▶ *Sociology* This develops understanding of how people interact with each other, and of diverse cultural issues, such as how using computers affects people's behaviour.
- ▶ *Linguistics* This concerns the syntax and semantics of language and can have a bearing on types of textual input and output.

In using the findings of these disciplines, an understanding of a range of other disciplines, such as ergonomics (the study of relationships between people and their working environments), engineering, design and (of course) computer science, is also needed.

In the face of such complexity, the question arises: 'How do we design good interfaces?' There are, broadly, two answers to this:

- 1 A body of guidance for HCI design has been developed. Design guidelines and principles are statements of good practice in designing interfaces; for example:

Be consistent Ensure that the appearance of the interface, and the manner in which it is used, applies consistently right across the interface.

Some key design principles are described in the next section.

- 2 If you are a software developer, it is important that you find out as much as possible about the people who will be using your software; that is, you should conduct a **user needs analysis**. In such an analysis, you would need to consider the following questions.

- ▶ What is the typical user like? For example, an interface that is appropriate for young children (large pictures and simple language) might not be appropriate for rocket scientists. Might some users have any particular requirements: might they have visual impairments, for example?
- ▶ What does the user want to do? For example, the requirements for an ATM are likely to be different from those for a home-banking system.
- ▶ Where will the system be used? Continuing the banking example, the ATM needs to be easy to learn, because a customer is unlikely to want to read a user manual while standing outside the bank!

Each of these aspects of a user needs analysis will be discussed further in *Unit 12*.

SAQ 4

- (a) Why is it necessary to understand the users when designing an interface?
- (b) What aspects of users are particularly important?

ANSWER.....

- (a) The behaviour of users can be difficult to predict, and designers cannot assume that users will react as they themselves would to a particular interface.
 - (b) To maximise the interface's usability it is particularly important to understand:
 - ▶ what the typical users are like;
 - ▶ what tasks they wish to carry out;
 - ▶ the environment in which they will use the system.
-

In this section, you have been introduced to the concept of socio-technical systems, and you have learnt about three aspects of a user interface. Having looked briefly at the history of user interfaces, the section ended by introducing some of the considerations that are important in user-centred design.

In Section 4, you will learn about some specific HCI design principles.

4

Introduction to user interface design

It would be very useful if there were a set of statements of good practice that could be applied to the design of any kind of interface. Attempts have been made in this direction, but it is very challenging, because systems and users vary so widely.

However, there is some generally accepted good practice that many designers do, in many circumstances, adhere to. For example:

- 1 use evenly spaced text with a ragged right margin, since it is easier to read than justified text with variable spacing between the words;
- 2 restrict the amount of information presented – only present what is necessary to the user.

The level of detail in these two statements is significant. The first one is easier to interpret and implement than the second: it is much more specific and dependent on the particular interface that is being developed. It is an example of a **design guideline**. Design guidelines are usually detailed and numerous, and cover such context-dependent issues as text formatting. They are typically relevant to specific kinds of system or specific contexts, and not to other kinds of system or other contexts. For example, text formatting is irrelevant in a telephone voice menu system.

Design principles are more abstract than guidelines, and should apply more or less universally to any kind of system in any context. Because they need to apply widely, principles are far less numerous than guidelines. The second statement above is an example of a design principle. In applying it, further questions have to be asked, such as: 'How do we judge what information is necessary?' These additional questions can be difficult to answer, and each person's response will probably be different. Thus, when using design principles, there is a significant element of subjectivity and choice in the design of interfaces. The key to handling this choice is to make decisions based upon knowledge of the users and the tasks they wish to carry out. As design principles are more widely applicable than design guidelines, the emphasis in the rest of this unit will be on design principles.

In this section we:

- ▶ introduce three important design principles;
- ▶ discuss the fundamental idea of metaphors (including direct manipulation).

The introduction of these subjects here will give you a practical understanding of some user interface issues and so provide a context for the material in the rest of this unit and in *Unit 12*.

In *Unit 12* we will introduce some additional design guidance.

4.1 Design principles

The leading cognitive psychologist and design expert, Don Norman, researched into the problems that people experience using everyday objects, such as doors, lifts, cars and video recorders. From this he abstracted some general design principles, which we will express as follows.

There is some debate in the HCI world as to whether good affordance and good visibility are 'principles', and exactly how these concepts should be applied to user interfaces. However, these concepts are useful tools for starting to analyse user interfaces, and you should have an awareness of at least one way in which they can be applied.

A user interface should have:

- ▶ good visibility;
- ▶ good affordance;
- ▶ good feedback.

To help explain these ideas, consider an office block which contains three lifts side by side. On each floor there is a light above the door to each lift. A person wants to travel in a lift, so they press the button situated by the lifts (there is only one for the three lifts). The light above one lift comes on and then goes off. Similarly the light above another lift comes on and then goes off. However, none of the doors opens. Finally, two of the lights come on together, and the user then realises that the lights indicate movement, not the arrival of a lift.

Visibility

How did the lift user know what the effect of pressing the button would be? The positioning of the button by the lifts suggests that the button has something to do with the lifts, but unless it is labelled, there is still some uncertainty: it could be an emergency stop button. **Visibility** is the extent to which something's purpose is apparent, that is, how clear it is what the thing is for. In user interfaces, as well as lifts, visibility is often affected by:

- ▶ labelling;
- ▶ position.

Look at the window in Figure 7. You should recognise the vertical bar at the right-hand side of the window: it is a scroll bar. How clear is it what the scroll bar is for? It may initially not be apparent what the effect of manipulating the bar would be. However, the positioning of the scroll bar as part of the window should suggest that the scroll bar will have a direct influence upon the window: exactly what would be determined only by trial and error.

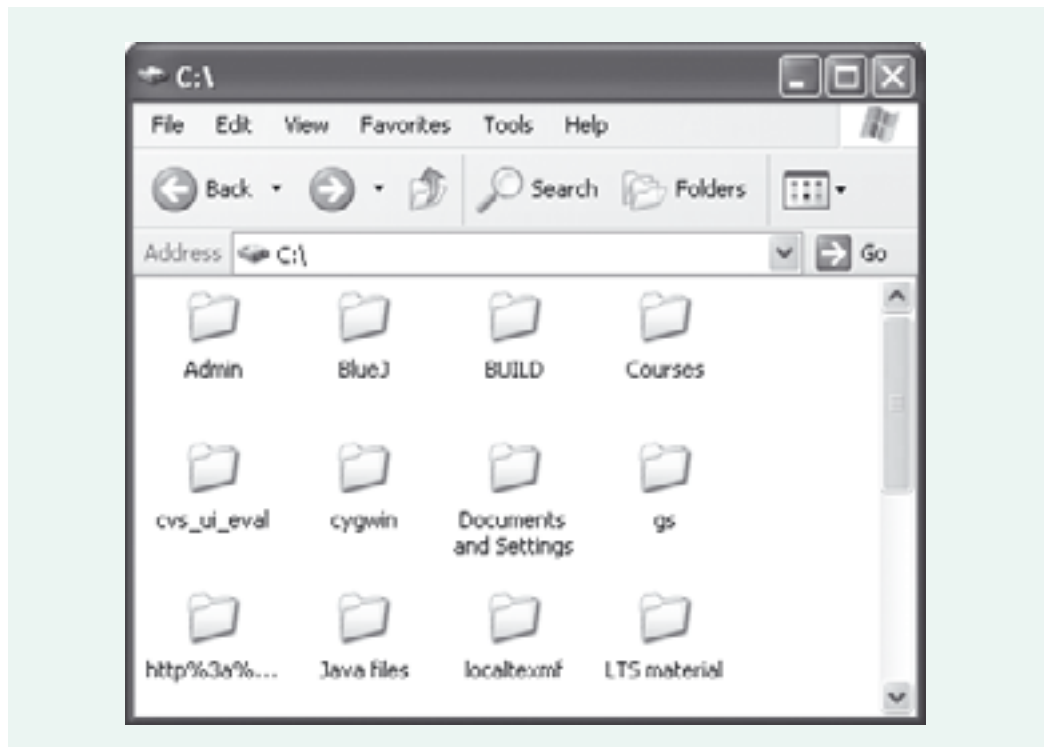


Figure 7 An example window

It is important to recognise that the meaning of *visibility* in the context of interfaces is a specialised, technical one. A widget (or a group of widgets) may be highly visible, i.e. conspicuous (the ordinary English meaning of visible), and yet have poor visibility in the technical sense. To take a trivial example, consider the button in the screen shown in Figure 8. The button is highly visible in the sense that it is easily seen.



Figure 8 A visible button with poor visibility

SAQ 5

Comment on the visibility of the button in the window shown in Figure 8.

ANSWER.....

The visibility of the button is poor because there is nothing to indicate what the button is for.

Affordance

We can develop the lift example: the user had to *press* the button, but how did they know this? The button may have had the following properties:

- ▶ it was raised from the surface of the wall;
- ▶ it was a typical button shape;
- ▶ it was labelled appropriately – ‘Press’, for example.

These properties of the button suggested to the user that it should be pressed. If the button had been flush with the surface (as many buttons now are), or had been a different shape (a star?), or had been unlabelled, the user might have been less sure about what to do.

We shall take the **affordance** of something to mean the manipulations that it appears to allow, that is, the actions that it appears should be carried out by the user to manipulate that thing. So we could say that, in the above circumstances, the lift button has affordance for pressing, or it *affords* being pressed.

Issues surrounding the original HCI concept of affordance, with a host of case studies, are examined in the book *The Design of Everyday Things*, (Norman, 2002). Note however that some of the terminology (including affordance itself) has taken on a range of new meanings in HCI.

Turning now to interface widgets, a button drawn onscreen by software cannot usually be physically pressed, by hand, in the way a lift button can. Instead, if the screen button looks like a real-world button, users may perceive that they should click on the screen button, simulating the act of pressing a real-world button. That is, the screen button affords clicking, just as the lift button affords pressing.

To say that a widget has good affordance, then, means that its affordance matches the way in which the widget should actually be used. That is, it is apparent how to use that widget.

Notice that affordance may depend on the user: one user's perception of how to operate a widget may not be the same as another user's perception. We will return to this important point shortly.

Referring back to Figure 7, the scroll bar is operated by dragging the block in the middle up and down using the mouse (or by clicking on the arrows). This action is suggested to the user by the arrows, by the raised appearance of the block (the dark lines around the block suggesting a shadow) and by the parallel lines that indicate it should be moved between them. Thus the scroll bar's appearance affords the scroll bar being used in the correct manner.

SAQ 6

Comment on the affordance of the button in Figure 8.

ANSWER.....

The affordance is good. The button should be clicked (as directed by the label), and it does indeed afford clicking since it has shadowed edges, giving the impression of a raised button that should be pushed. Thus it is clear how this widget should be used.

Feedback

Finally, **feedback** refers to information about the results of the user's action. In the lift example presumably the button 'depressed' in response to the user pushing it – otherwise, they might have thought the lift was broken. Indication of movement between floors is also relevant information, but not if it is misinterpreted as indicating lift arrival. In general, feedback should be of an appropriate nature, should be relevant and consistent, and should help the user rectify any mistake.

Returning to Figure 7, the feedback from manipulating the scroll bar takes the form of a changing set of icons appearing in the window, and the block appearing to move (i.e. a change to the widget itself).

Exercise 1

Doors in homes or offices commonly have one of the following kinds of 'handle':

 (a handle) and  (a flat metal plate).

Consider how you use these 'handles'. What problems can arise with visibility, affordance and feedback?

Discussion.....

Visibility

The purpose of a flat metal plate is perhaps less obvious than that of a protruding handle: a flat metal plate might be perceived as a merely decorative adornment to a door, rather than an object with a purpose. However, given its location on a door, most

people would have a good idea of what each kind of handle is for, especially as they are prevalent in homes and offices.

Affordance

A handle such as  affords holding and being pushed down, which is indeed how it should be used. The handle looks comfortable to hold, and its shape correctly suggests that the user's hand should go on top. In this respect its affordance is good. However, it may not be clear whether, having pushed the handle down, it should then be pushed or pulled to open the door.

With a flat metal plate there is no problem of knowing whether to push or pull: it can only be pushed.

Feedback

If the door is manipulated in the correct way, there is clear feedback: the door opens (and, with a handle such as , the handle moves under the pressure of the user's hand). But if you push a door and it does not open, it is likely that no feedback is given: you will not immediately know whether the door is locked, or whether the door should be pulled instead of pushed.

SAQ 7

- What is the difference between the affordance and the visibility of a GUI widget (or group of widgets)?
- Briefly describe what is meant by good feedback in the context of a GUI.

ANSWER.....

- Affordance is concerned with what to do to 'work' a widget (click it, sing to it, drag it, type in it, read it), while visibility is what the widget is for (adjusting the volume, switching off, rewinding).
- Good feedback is where the interface clearly and consistently indicates to the user the result of the action they have just carried out. It also includes giving information to users about how to correct any mistakes.

4.2 Reflecting on the user

Above we presented some key ideas about the design of user interfaces as if they were equally applicable whatever the characteristics of the users. In fact, as was mentioned earlier, design principles do not stand alone: they must be accompanied by an evaluation of the needs and characteristics of the user, in order to create a usable interface. It is often necessary to reflect on how different people might react differently to the same interface.

Suppose you see a small portable box (as in Figure 9) that has:

- ▶ two circular knobs;
- ▶ five push buttons;
- ▶ music coming out of it.

What is the small box for? You will almost certainly be able to guess its purpose. Not only that, but you will probably also have a good idea what will happen if you twist the knobs or press the buttons. You will guess how to look for different music, or alter the volume. And if you are unsure about exactly what a feature does, you will be able to work it out by trial and error.

Of course, your ability to understand the device in this way depends on your culture and the era in which you live. To someone with the cultural background of an early Victorian, none of this would be at all obvious.

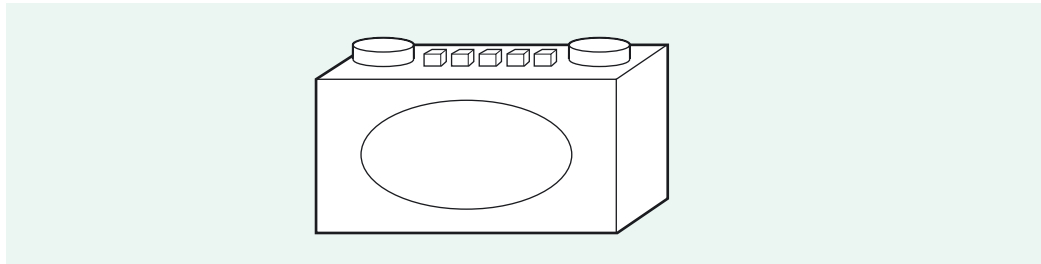


Figure 9 Illustrating learnt affordances

To understand the effect of the person, we will concentrate on affordance in the above situation. Even for the early Victorian, the radio with its various knobs and buttons would still have certain affordances. To any human with experience of simple machines and toys, the dials suggest the possibility of twisting or perhaps pulling. They fit the fingers neatly and are 'just asking' to be twisted. Similarly, the buttons invite pushing or pulling.

To some extent these affordances will apply to any human and perhaps to some other primates. On the other hand, to very small children, the controls may provide good affordance for putting in the mouth. To hooved animals they will not provide much turning or pushing affordance. But the radio might provide them with good affordance for kicking. To people with some physical disabilities, the controls might provide little affordance. To a very angry human, the radio might have an affordance for throwing across the room. These examples all illustrate the fact that the same thing or interface can have quite different affordances for different people in different circumstances.

Originally the term affordance was restricted to physical objects and the innate, unlearned responses they elicit. However, the term is now often used in HCI to include culturally learnt responses as well. One way to acknowledge the different uses of affordance is to use the term **learnt affordance** in the case of culturally learnt responses: not a property of a thing alone, but a property of the combination of the thing and a person with given skill, capabilities and training who is from a given background. However, for simplicity, we will use the term *affordance* to cover the whole spectrum.

Exercise 2

For each of the GUI widgets below, as shown in Figure 10, contrast the affordance it offers a novice computer user with the affordance it offers an experienced computer user.

- (a) The Save button.
- (b) The File name box (the one containing the text 'Dear Ray.doc').

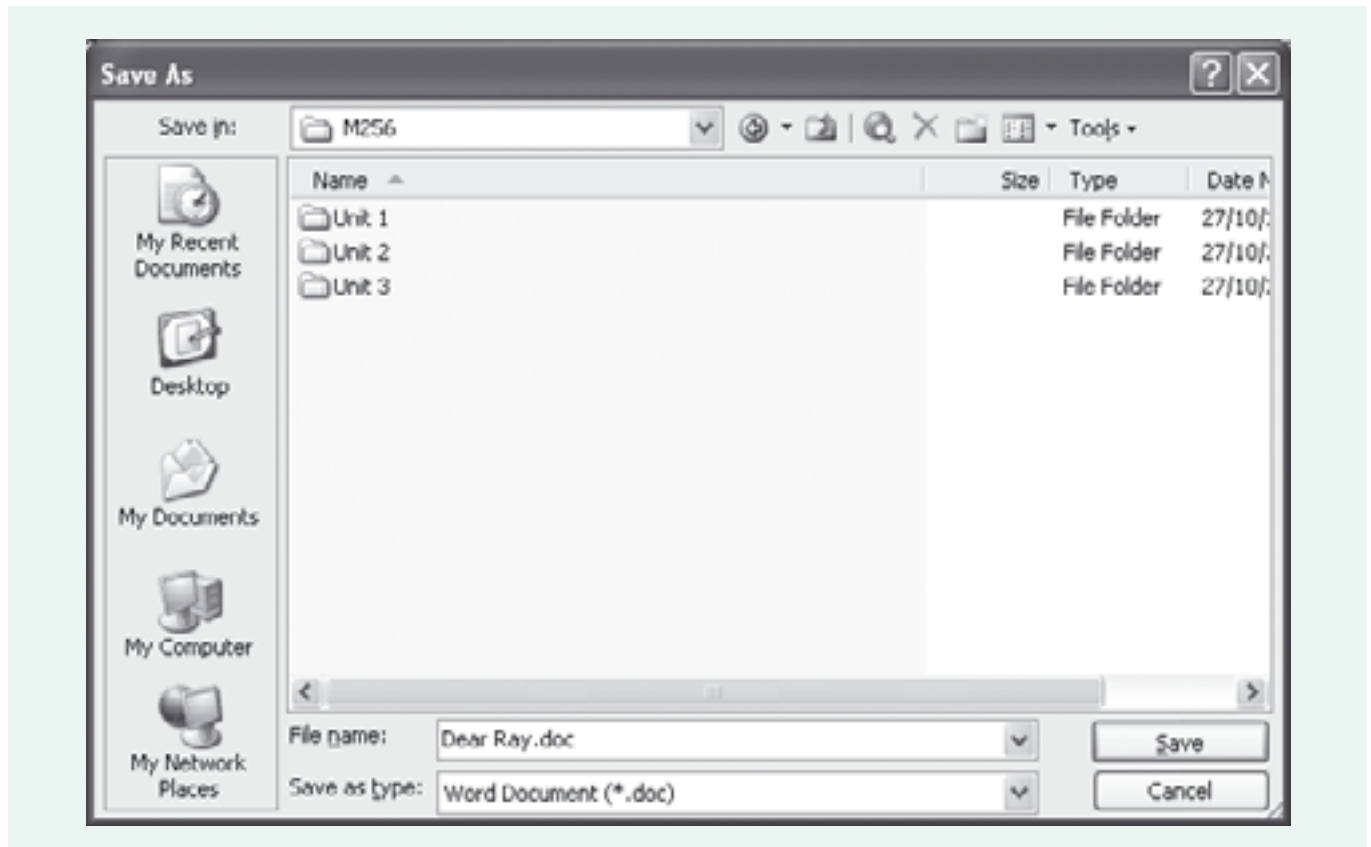


Figure 10 Saving a file

Discussion.....

- (a) The **Save** button should, of course, be pushed (clicked). It has shadowed edges, giving it the raised appearance of a real button. Even a novice user is likely to infer correctly from this that the button should be pushed. (However, the fact that the **Save** button looks different from the **Cancel** button might confuse novice users. Should they operate the two buttons in different ways?)
- (b) Clicking on the downward-pointing arrow at the right of the **File name** box results in the appearance of a list of items, from which the user can select. An experienced user is likely to correctly infer that they should click on this arrow, whereas a novice user might not.

In fact, the **File name** box can also be typed in. The box appears indented, which suggest to an experienced user that typing is possible. Novice users, however, may well question how to use this box.

This kind of widget is sometimes called a *list box*.

Exercise 3

Visibility also can depend on the user's background and expectations. Can you think of a widget that illustrates this?

Discussion.....

Here are some widgets you might have thought of.

- 1 A user familiar with 'cut and paste' may recognise the purpose of a button labelled with the icon in Figure 11. However, to someone who has not used cut and paste, visibility will be low.



Figure 11 A common icon

- 2 Text labels will not give good visibility for a user who does not understand the language used.
- 3 The purpose of a ruler along the screen in a word-processing application will be obvious to those who have used a typewriter, but others may not have any idea what it is for.

There is much more that could be said about visibility, affordance and feedback – far more than is appropriate for an introductory course such as M256. However, you now have the beginnings of a toolkit which you will apply in the rest of this unit, and which will be developed in *Unit 12*, where you will start to actually design interfaces.

In the discussion for Exercise 2(a) we said that even a novice user might correctly infer how to use a button widget, through their understanding of how real-world buttons work. This leads to another important tool in the user interface designer's toolkit: user interface metaphors.

4.3 User interface metaphors

Metaphors have traditionally been linguistic devices. Thus, someone may refer to a 'glaring error', 'food for thought' or 'the sunset of one's career'. None of these examples should be interpreted literally: errors do not shine in a dazzling way; thinking does not involve eating; and a career does not have a sunset. However the use of the metaphor adds expressiveness and draws in knowledge of other comparable situations with which the speaker and listener are familiar. **User interface metaphors** work in a related way.

For example, Figure 12 shows the interface to a calculator application, supplied with a version of Windows.



Figure 12 A calculator metaphor

This interface metaphor assumes that the user is able to recognise and use a simple hand-held calculator. In a well-thought-out metaphor such as this, possessing some specific existing knowledge, a **mental model** (often of the physical world), results in the user being able to infer how to operate the interface.

Thus a user who is familiar with a simple hand-held calculator should be able to work out that clicking on the buttons is equivalent to pressing the buttons on the calculator, and they should also realise that **sqrt** is an abbreviation for 'square root'. Thus we have a user interface that most users will be able to use without any training.

The central idea behind a user interface metaphor is that the designer searches for some relevant pre-existing detailed knowledge, likely to be a mental model common to users, of some other system or real-world object or situation. For example, many users will know about address books, diaries, radios, CD players, recording devices, maps, telephones and calculators. The technique, then, is to design a user interface that lets the user make use of this pre-existing knowledge. A well-chosen metaphor, applied consistently, will offer the designer many opportunities to provide good visibility, affordance and feedback, since the mental model will provide numerous hints to users about how to operate widgets, what they are for, and what kinds of feedback to expect.

The choice of metaphor is extremely important, as is the precise way it is applied. For example, a complex scientific or graphical calculator could have been chosen. This would be functionally more powerful, but it might confuse users unfamiliar with the hand-held version. Thus it is centrally important to consider the profile of users for whom the interface is being developed.

On most basic calculators, however, the division button is labelled with the $\div$ symbol, but in Figure 12 the / symbol is used, so it may not be obvious to those with little experience of mathematics (or computer programming) that the / symbol is for division.

Exercise 4

Can you work out how you would perform the calculation $3/9 + 4/7$ on the on-screen calculator? What additional knowledge of calculators do you need in order to carry this out?

Discussion.....

You would perform the calculation on the on-screen calculator by using the key presses: $3 / 9 = M+ 4 / 7 + MR =$. This requires a knowledge of calculator memories (M+ means add result to memory, and MR means memory recall). Would the user have this additional knowledge? If you did not have this knowledge, do not worry, it just reinforces the importance of knowing as much as possible about the users of the interface.

We have used the Standard version of the calculator. The Scientific version of the calculator applies precedence rules, so the key presses would be: $3 / 9 + 4 / 7 =$.

Exercise 5

Consider the user interface in Figure 13 and critique it (that is, make a careful evaluation of it) according to:

- the consistency of the interface with any mental models;
- whether and how any metaphors employed contribute to visibility and affordance.

Note the following:

- The time can be set using the list box below the clock dial. A new time can be entered by typing, or the hour can be changed by clicking the up and down arrows at the side.
- The date can be set by selecting a month, a year (several years behind, or in advance) or a day. The corresponding day is automatically highlighted on the calendar.

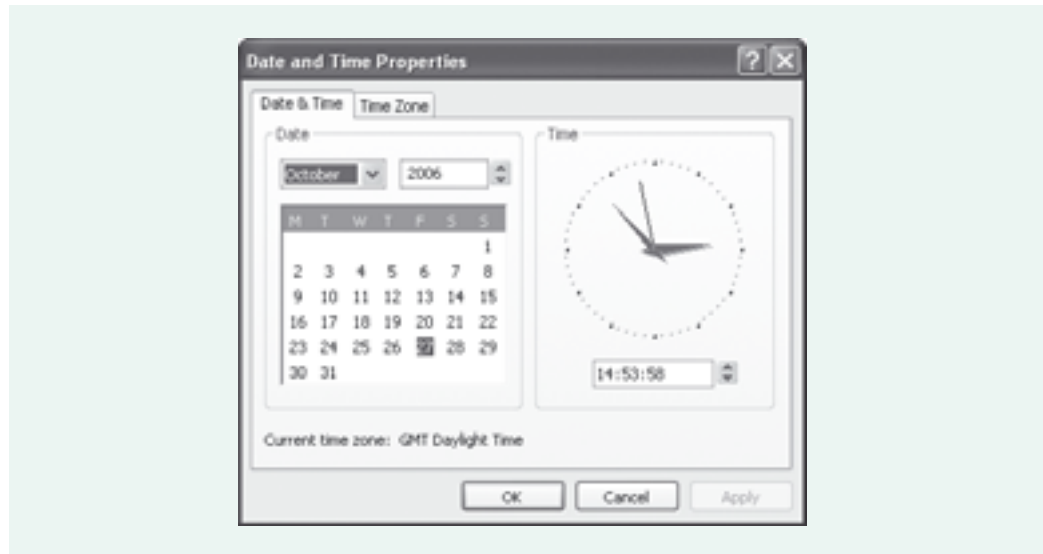


Figure 13 System date and time display

Discussion.....

- (a) This is an interesting interface, about which you could make many sensible comments relating to metaphors. Here we present a few key points.

There are (at least) two mental models involved in the **Date & Time** display: a calendar; and a clock (or watch).

The purpose of the interface is primarily to display information about the current date and time. With the calendar and clock metaphors this information should be easy to understand, for any user who has experience with these mental models. The clock metaphor has added value, in that both a clock face and a digital display are given – sometimes, but not often, the case with real clocks and watches.

- (b) The user can manipulate the display in two ways.

- ▶ Setting the time using the list box below the clock dial. This has some similarities with the way digital clocks are set, and visibility and affordance are thereby enhanced, but there is an inconsistency which may confuse users: it is not possible to alter the clock face by directly manipulating the clock hands – something that is of course possible with certain clocks.
- ▶ Selecting any month, year and day. Using a paper calendar or diary you are likely to have to manually search forwards or backwards through the intervening days to select a day of interest. The additional interface functionality, whereby the days of the month are automatically available for selection when a month is selected, seems useful and unlikely to confuse users, since the widgets (the list boxes etc.) have inherently good visibility and affordance.

The desktop metaphor

Most GUI interfaces combine separate widgets to create a single, coherent environment. Very often this environment uses a desktop metaphor. Thus the background corresponds to the desk, and the various windows and icons correspond to the clutter on most desks. Figure 14 illustrates such a desktop.

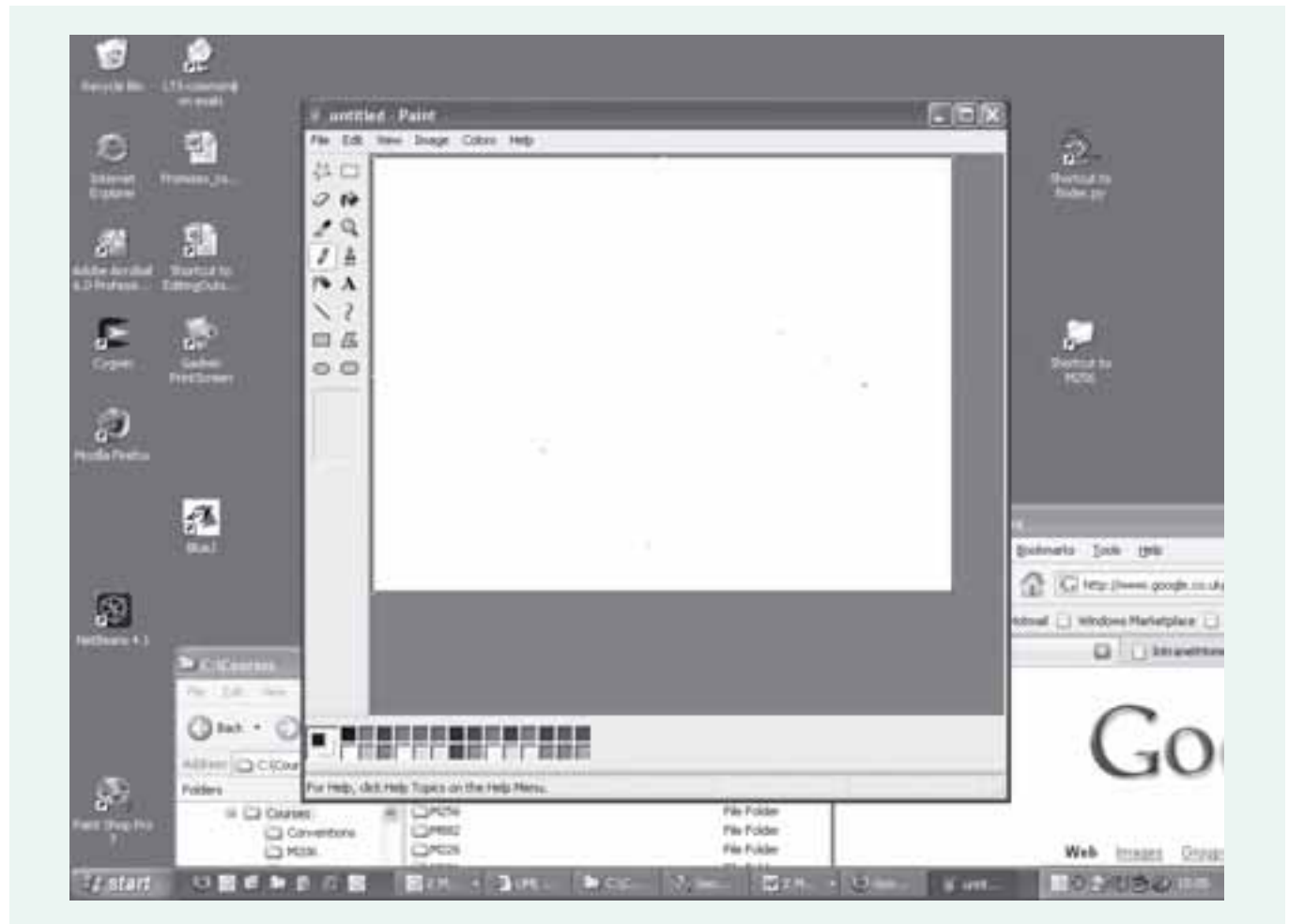


Figure 14 A desktop

With the desktop metaphor the parallel with a real desk works up to a point, but the functionality of the electronic desktop is much greater: not only can you draw on this desktop (using the Paint application), but you can also access your filing cabinet (for example, search through the hard disk). How many desks have a filing cabinet on top of them? Similarly, you can search the Web – there seems to be no real-world analogue of this activity, short of having a vast audio, video and print library on your desk (together with a telephone and video conference facility)!

In fact, it is hard to think of a situation in which a user could work out what to do by appealing to their experience of real desktops, and there are several inconsistencies, as noted above, but this does not bother users. The essence of the metaphor is the idea that all the necessary equipment is laid out in front of the user. This is all that is needed: if designers did follow the metaphor more literally, say by putting the wastebasket under the table, it would make the interface harder to use, not easier. Users generally find interface metaphors helpful even if the parallels between the mental model and the application are quite loose.

Of course in many cases the correspondence is so close that the pre-existing mental model will be enough to inform a user how to use the application, e.g. a calculator, a media player or a diary. Other metaphors are just as close, but not necessarily perceived as such by users. A word processor is a good example: the metaphor is a sheet of paper in a typewriter. The ruler mimics the typewriter ruler, and people who have been familiar with manual typewriters will know what the ruler is for and how to use it without being told. However, a user without this background might not realise what the ruler is for. In fact, such a user might even be unaware of its existence.

It is important to recognise that user interfaces that use metaphors are often forced to behave partly like other parts of the user interface, and partly like the real-world analogue. The calculator metaphor may be quite effective, but you do not find physical calculators with menu bars at the top. Even less do you find calculators with a button in the top right-hand corner that will make them disappear! On the other hand, a user interface whose window cannot be managed like other windows would, in all likelihood, be a source of great problems to the user. Where a user interface uses a given metaphor, boundaries with other parts of the system that do not use that metaphor may need careful attention to avoid possible confusion.

SAQ 8

Choose three user interface metaphors from this course or from your own experience. Identify the mental model the user is expected to be familiar with, and the application (i.e. program) to which the metaphor is applied. For example:

Mental model	Application
CD player	on-screen MP3 player

ANSWER.....

Here are some other common metaphors. You might have thought of three of these, but there are many other possibilities.

Mental model	Application
filing cabinet	file system
calculator	calculator application
drawing board	paint application
clock/calendar	system date and time display
button	action button (within many applications)

Direct manipulation

The notion of physical manipulation gives rise to one of the most pervasive user interface metaphors, adopted to varying degrees by many common user interfaces. One aspect that such interfaces have in common is that when the user issues a command they apply it 'directly' to representations of 'objects' that the command has to change or use.

For example, most users are very familiar with clicking on icons to run applications, and dragging icons between windows to copy files. This is one aspect of a style of interaction referred to as **direct manipulation**. Direct manipulation conforms to, and also extends, the metaphor based on the mental model of people in the real world moving files about, pressing buttons, sliding volume controls and so on.

Above we were careful to use the term *representation*. Throughout the course we have made the distinction between the core system and the user interface. In direct manipulation the user interface often contains visual representations of some of the software objects that make up the core system. When the user manipulates one of these representations, in reality underlying software is generally causing changes to be made to the core system, or retrieving information from the core system.

Apart from applying commands directly to representations of objects, other aspects of direct manipulation are as follows.

- ▶ The representations of the objects and actions of interest should be continually visible on screen. Files and applications might appear as icons, actions as buttons and so on.
- ▶ Instead of typing in commands, 'objects' should be manipulated directly in ways that seem natural and obvious, e.g. by pointing, clicking and dragging.
- ▶ Actions should be carried out rapidly and incrementally so that the interface appears responsive and the effect on objects is immediately visible. Also, the actions should be reversible so that mistakes can be rectified.
- ▶ 'Small' operations should be used incrementally to cause 'big' effects. For example, the 'small' operations of highlighting (selecting) a file icon followed by dragging to a folder have the 'big' effect of moving the file.

Many benefits have been claimed for direct manipulation. For instance:

- ▶ novices are able to learn more quickly;
- ▶ fewer user errors are made;
- ▶ users can see if their actions are furthering their goals and, if not, change what they are doing.

It is hard to be precise about what does and does not constitute a direct manipulation interface, and there is not universal agreement about which of the claims are borne out and to what extent. However, in general, direct manipulation is held to be a valuable interface technique.

Direct manipulation is implemented most directly and straightforwardly when visible objects correspond to objects in the software, and labelled commands on menus and buttons correspond to messages. However, object-oriented technology alone does not yield good interfaces: direct manipulation interfaces can easily be designed poorly and inconsistently.

Exercise 6

Give an example of each of the following, justifying your choice.

- (a) Direct manipulation when using a word processor.
- (b) Direct manipulation that applies to a window.

Discussion.....

- (a) When you are using a typical word processor, the objects of interest (the text) are continuously represented. There is instant feedback as you type new text. Cutting and pasting of text is done with a physical gesture for selection, and then by pressing labelled buttons. Selections are made instantly visible by highlighting. There are extensive undo and redo functions.
 - (b) A physical action can be used to move the window, with instant feedback. Another action can be used to resize a window, with instant feedback. However, the user must know the convention of where to 'grasp' the window to achieve these effects.
-

Consider the Solitaire (Patience) game that comes packaged with Windows. Figure 15 illustrates part of a game.

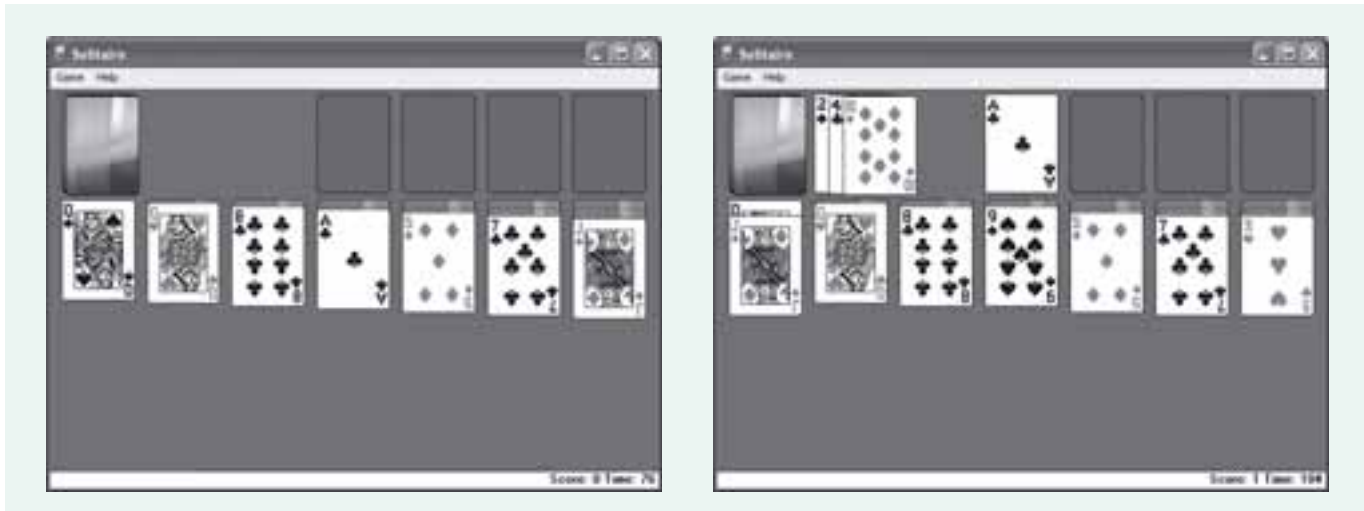


Figure 15 Part of a game of Solitaire: (a) start position; (b) after several moves

To reach the second position, it was necessary to:

- ▶ drag the Ace of Clubs to its new position at the top;
- ▶ drag the Jack of Diamonds to just underneath the Queen of Spades;
- ▶ click on the piles previously topped by the Ace of Clubs and the Jack of Diamonds;
- ▶ click on the top of the deck of cards at the top left corner.

Using direct manipulation in this game is easy and largely intuitive. Imagine the alternative situation where, instead of direct manipulation:

- ▶ commands had to be entered using a special language;
- ▶ the objects were not visible;
- ▶ the feedback was via text responses.

A game might look something like this:

aceclubs	Moveto:	pile1
jackdiamonds	Moveto:	queenspades
pile8	ShowNextCard	
pile11	ShowNextCard	
drawdeck	ShowNextCard	

This approach would be satisfactory for simple examples, but for a whole game it would be tedious and error prone.

Exercise 7

Critique the Solitaire game, judging it according to the metaphor involved and the principles of good visibility, affordance and feedback.

Discussion.....

The representations of cards look very much like real cards. The edges are slightly darkened, suggesting depth and hence that they can be moved in some way. Also, in piles, the cards are represented as being distinct from one another (they overlap, but do not obscure each other completely). This helps to identify them as separate entities. Thus they do give the impression that they should be moved (good visibility), but exactly how is not very clear (affordance is less good). However, once the concept of dragging cards has been grasped, this is applied very consistently.

The feedback is excellent: it is easy to see, by the changed position of a card, that a move has been successful.

If you ever play this game, you will also find that the application provides feedback when you attempt an illegal move: it will not let you make the move! This is a minor inconsistency with the actual card game, where there is nothing to stop you cheating. However, this could be considered to be added value.

Overall, the metaphor adheres closely to the card game as played with an actual pack of cards, providing a usable and easily learnt interface.

SAQ 9

What is the relationship between direct manipulation and the core system?

ANSWER.....

The elements that make up a direct manipulation GUI interface are often visual representations of some of the objects that make up the core system. Manipulation of these elements results in appropriate messages being sent to the core system and, depending on the messages, the objects are changed or provide information.

In this section you have been introduced to the HCI design principles of good visibility, affordance and feedback, and seen how they are affected by the background, knowledge and experience of the user. You have also learnt how user interface metaphors can enhance visibility, affordance and feedback, and, in particular, how pervasive the metaphor of direct manipulation is.

In the next section, you will have the opportunity to apply these design ideas in the process of reviewing common interface widgets.

5

Widget functionality

In Section 4 you were introduced to some design ideas connected with GUI interfaces, specifically: affordance; visibility; feedback; and user interface metaphors (including direct manipulation).

Of course, when designing a GUI interface, a designer needs to consider not only ideas such as these but also the fundamental question of what types of **functionality** different kinds of widget offer. That is, what types of task do they enable the user to do, and what types of communication between user and system can they mediate?

The main aims of this section are to:

- ▶ briefly review a small number of common kinds of widget, concentrating on their associated functionality;
- ▶ give you the opportunity to practise applying the design ideas mentioned above.

In *Unit 12* we will follow up this work by describing how a designer might select and combine widgets into a complete GUI interface.

We have already mentioned many of the widgets that will be described in this section; we expect you will be familiar with others from your own experience as a computer user. Those you will look at in this section have been taken from the Windows XP operating system. Thus, if you are using a different version of Windows or another operating system, the widgets may look slightly different.

5.1 Menu bars and menus

From the earliest days of graphical user interfaces the **menu bar** was almost universal, and today there are almost no applications which do not display a menu bar, often along the top of the application window.

You should be familiar with the fact that a menu bar provides a way of hierarchically grouping commands. The words along the menu bar represent major groupings of commands, arranged in **menus**. The menu name suggests what kinds of command may be found. Figure 16 shows a window: the bar that contains the words **File**, **Edit**, **View** etc. is the menu bar, and the **View** menu is displayed. It can be described as a *pull-down* menu since it appears to be pulled down from the menu bar.

The first two menus, **File** and **Edit**, are typically present in every Windows menu bar; each application will have additional menus depending on what the application is designed to do. To look at a menu you simply select its name, and then you can select the option that you require from the menu.

Pop-up, or contextual, menus are also common. They are typically accessed by right-clicking on a part of a screen, or another widget. They provide shortcuts to particular tasks or information that the user may need in the particular context.

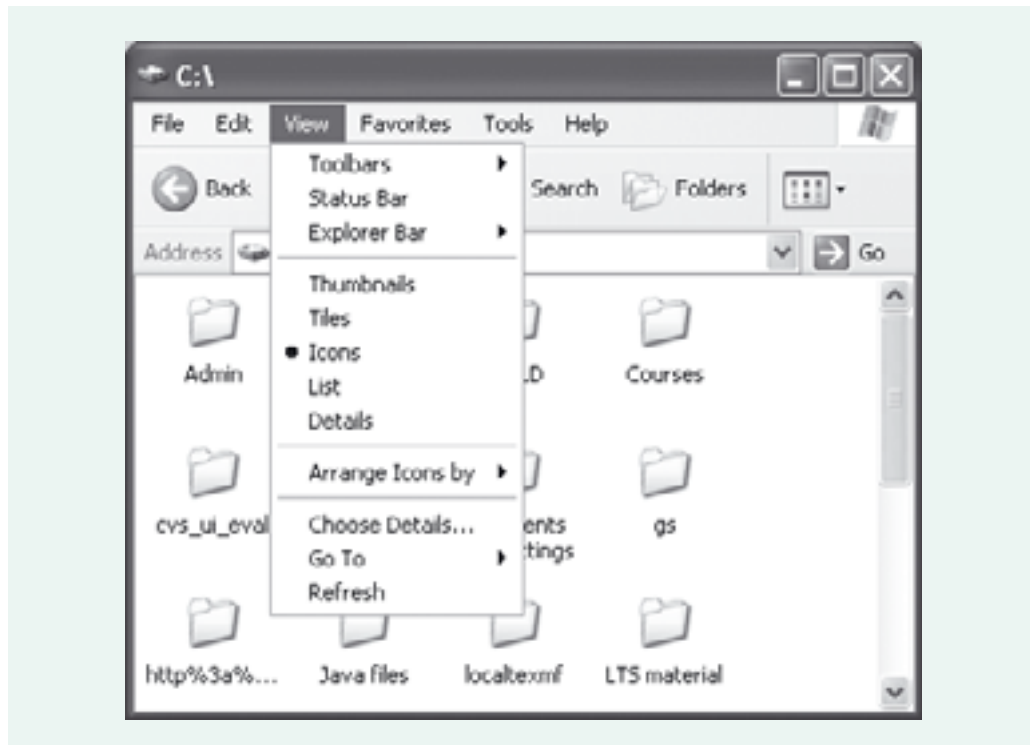


Figure 16 A menu bar and a pull-down menu

From an HCI perspective, a menu is a reasonable metaphor: you are presented with a selection from which you choose – just like the menu in a restaurant. However, there are a number of inconsistencies, as you might expect. For example, there is no obvious real-world analogue to the menu bar. You could consider the menu bar to be equivalent to a shelf in the restaurant on which the various menus can be found – the main courses, desserts, wines and so on. However, it is doubtful that this would be immediately apparent to a novice user, particularly given the lack of visual clues. Similarly, screen menus have unexpected functionality. For example, a customer in a restaurant does not press a button on the menu to order food, but this is effectively what happens on a screen menu. Thus screen menus are active, whereas restaurant menus are passive.

In terms of visibility and feedback, the menu in Figure 16 is good. The different options are clearly distinguished within the menu (although the wording could be more meaningful in places). When the mouse moves over an option, the option is highlighted, thus providing useful feedback. The feedback given on making a selection will depend upon which option has been chosen.

The affordance of the menu is less good: it is not immediately apparent that each option can be clicked on. Another point is that an option is selected by a single mouse click. There is inconsistency, which may be confusing: on a menu, a single click activates the chosen option; yet, for example, in the window itself, a double-click on an icon is needed to open a file or activate an application (a single click only selects it.)

Exercise 8

How might you improve the affordance of the menu in Figure 16?

Discussion.....

It would be possible to make the menu items look more like buttons (by shading the edges). This has the disadvantage of stretching the menu metaphor even further: restaurant menus do not contain buttons. But, importantly, it might make the user realise that only a single click is needed to activate them.

5.2 Buttons

Buttons are typically used to enable the user to initiate a single action. They are sometimes called **action** or **command buttons**. Clicking on a button causes an application to carry out the associated action. Buttons are usually labelled, to increase their visibility, either with text, as in Figure 17, or with an icon. Pop-up textual descriptions, appearing when the cursor is moved over the button, are sometimes used to give additional information (as they are with other widgets).

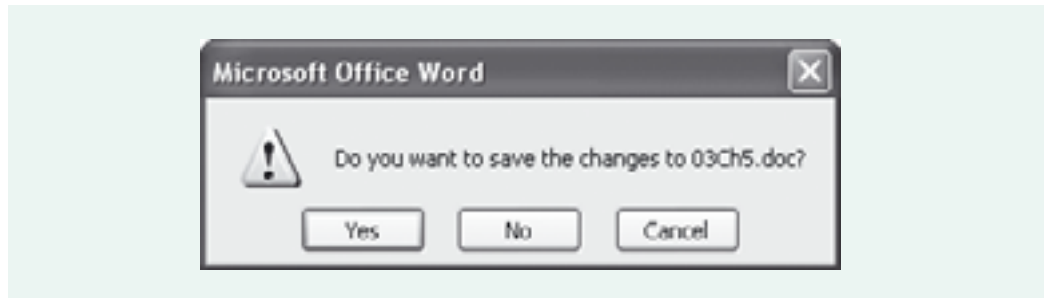


Figure 17 Buttons

Through the dialogue box in Figure 17 the user is being asked whether they want to save changes made to a particular document. The default is to save the changes (there is a thick line around the **Yes** button, which suggests that the button is raised higher than the others).

Exercise 9

Figure 18 shows some buttons that appear at the top of a word-processing application window.



Figure 18 Buttons in a word-processing application

Here is some outline information about these buttons.

- ▶ Clicking on the first button results in the user being offered a choice of documents to open.
- ▶ Clicking on the second button results in the current document being saved.
- ▶ Clicking on the third button results in the current document being printed.

- (a) Comment on the visibility of each button.
- (b) Concentrating on the second button, if you were the designer of the application, how might you ensure that you apply the principle of good feedback?

Discussion.....

- (a) The icons on these buttons are used in many applications. Thus, experienced computer users will understand their purpose. In detail:
 - ▶ The first button icon appears to represent a folder, and the arrow perhaps represents something being taken out of it – a previously filed piece of paper, perhaps. The metaphor of office filing systems helps here, though novice users may not initially know how to interpret the icon.

- ▶ The icon on the second is of a floppy disk, at one time the main kind of external data storage device. Novice computer users may be more familiar with storing data to other kinds of device, and may not understand this icon. Moreover, it is questionable whether a picture of a disk signifies saving something to a disk more than it signifies the notion of retrieving something from a disk, or for that matter ejecting a disk.
 - ▶ The icon on the third is of a printer, which is likely to be understandable to even novice users.
- (b) There is a variety of common kinds of feedback given when an action button is pressed. As a designer you might choose one of the following as immediate feedback:
- ▶ The button pressed might momentarily 'blink' and be highlighted by black edges, giving it the appearance of a pressed real-world button.
 - ▶ The button might momentarily change colour. This has a real-world analogue in buttons that light up when pressed.
 - ▶ If the resulting operation takes time, the button might remain unchanged but the cursor alter to an hourglass , to indicate that something is happening.

You might also find it appropriate to inform the users, using a dialogue box perhaps, of the results of their action: 'File 03Ch5.doc has been saved to C:\Units\unit3\D3' for example.

5.3 Radio buttons and check boxes

Radio buttons and check boxes are used for setting on/off states to be recorded by an application for immediate or subsequent action. Figure 19 shows an example of radio buttons.

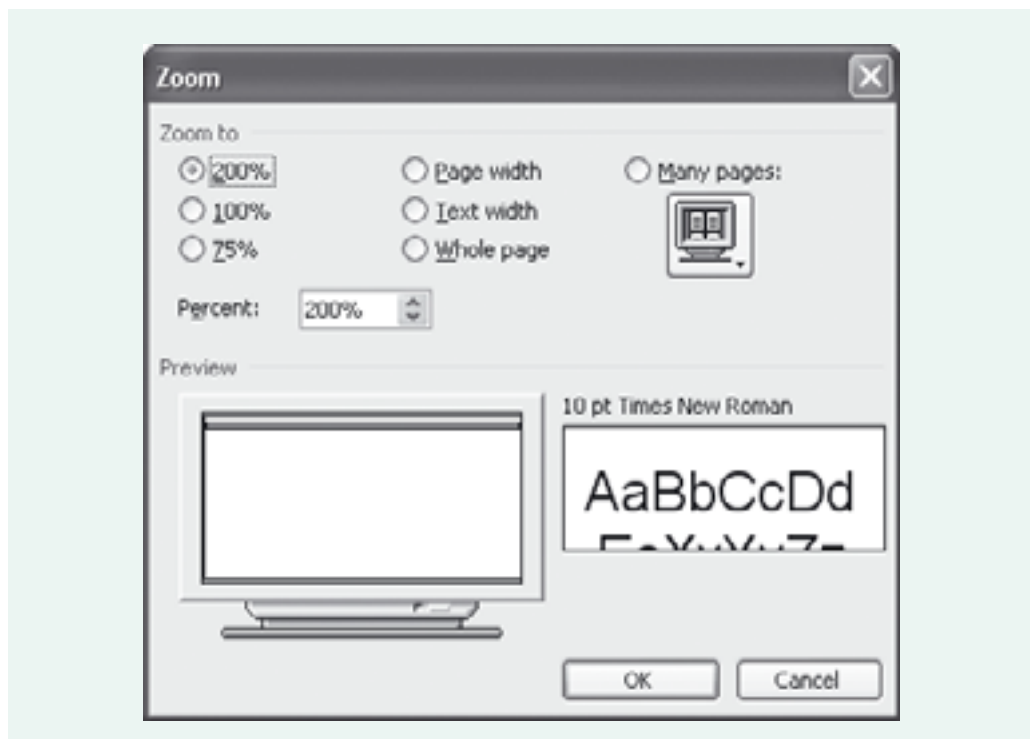


Figure 19 Radio buttons

Radio buttons are used where there is a set of choices, precisely one of which must be selected (that is, precisely one state must be 'on'). Clicking on a radio button to select an option causes the radio button to be highlighted and to remain so until another of the mutually exclusive options is selected. For example, in Figure 19 the buttons labelled 200%, 100%, 75% etc. are radio buttons: the document can be viewed only at one size at any one time. In the example, the viewing size is 200%.

SAQ 10

Radio buttons are usually presented in groups of two or more. Briefly explain why this is the case.

ANSWER.....

It is not appropriate for there to be a *single* radio button, as this is not compatible with a set of mutually exclusive choices. A radio button is only deselected when another one is selected, so if there is only one and the user selects it, they cannot reverse their choice.

Check boxes are used to select from a set of options which are not mutually exclusive (that is, any number of states can be 'on'). Thus check boxes are used where selection of a number of pieces of information (including none) is allowed. In Figure 20 the square boxes labelled Strikethrough, Shadow, Small caps etc. are check boxes. A tick indicates that the option has been selected.

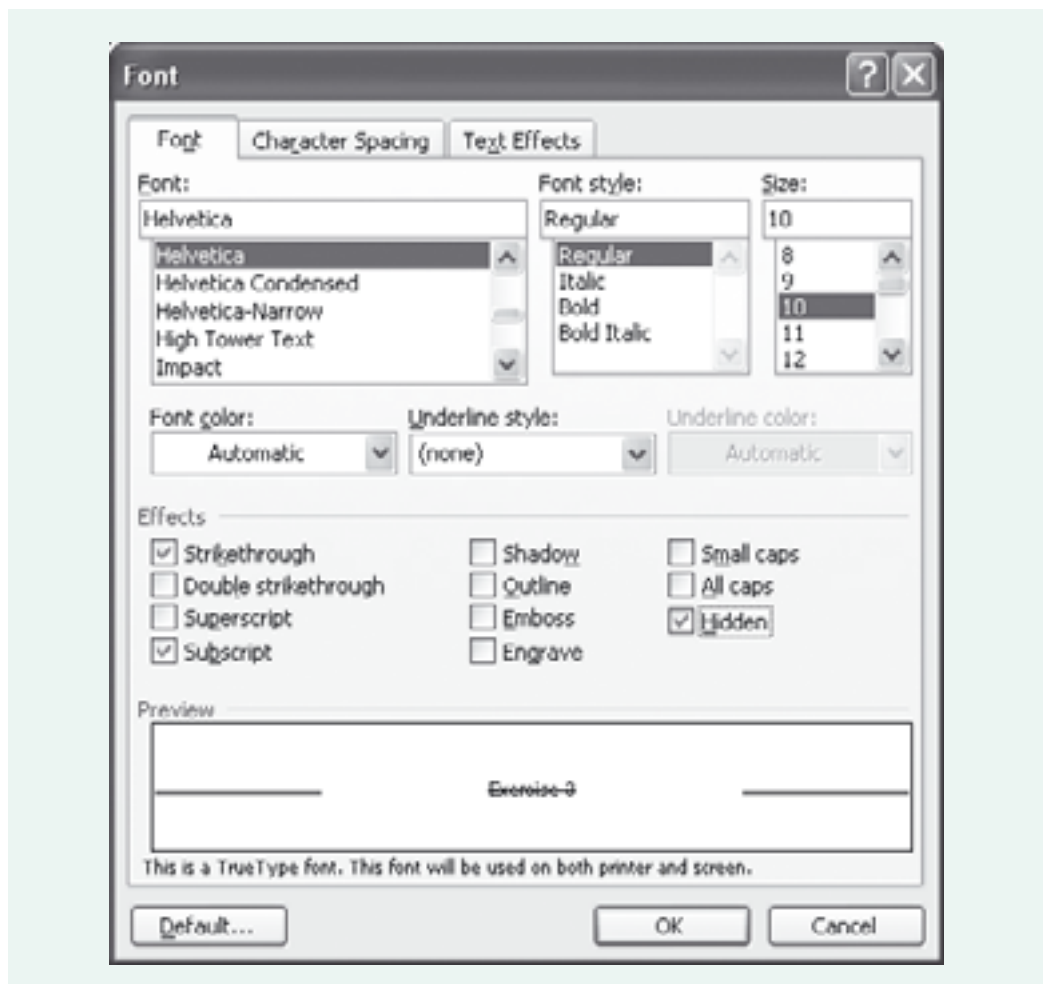


Figure 20 Check boxes

Exercise 10

- (a) What metaphor do check boxes employ?
 (b) Comment on the affordance of the check boxes in Figure 20.

Discussion.....

- (a) The most obvious metaphor is that of completing a questionnaire with a series of boxes corresponding to questions which the user leaves blank or manually ticks.
 (b) The check boxes look like indented blank squares, suggesting that something should be entered in them. It might be considered that the squares look as though they should be typed in, rather than simply clicked, which suggests that the affordance for some users might be poor.

SAQ 11

What kind of widget(s) would you recommend for the following purpose? Give a reason for your answer.

The user of an insurance system must indicate how long they have been living at their home address. One of the following options must be chosen:

- ▶ less than 6 months;
- ▶ between 6 months and 2 years;
- ▶ over 2 years.

ANSWER.....

A set of three radio buttons is well suited, because precisely one option must be selected.

SAQ 12

What kind of widget(s) would you recommend for the following purpose? Give a reason for your answer.

The user is to be given the choice of showing, or not showing, hidden files or subfolders within a particular folder.

ANSWER.....

Since there are only two choices in this case, a single check box would be appropriate. If the box were labelled 'Show hidden files and folders', a tick in the box would mean that hidden files and folders would be displayed, and no tick would mean the opposite.

Since the options of hiding and not hiding are mutually exclusive, you could also use a pair of radio buttons, each representing one option.

5.4 Windows and dialogue boxes

A **window** (or **screen**) typically gives a view onto some aspect of a system or application. Windows look different on different GUI interfaces, and on each GUI there may be various types of window each of which looks different. Figures 21 and 22 show typical example windows.

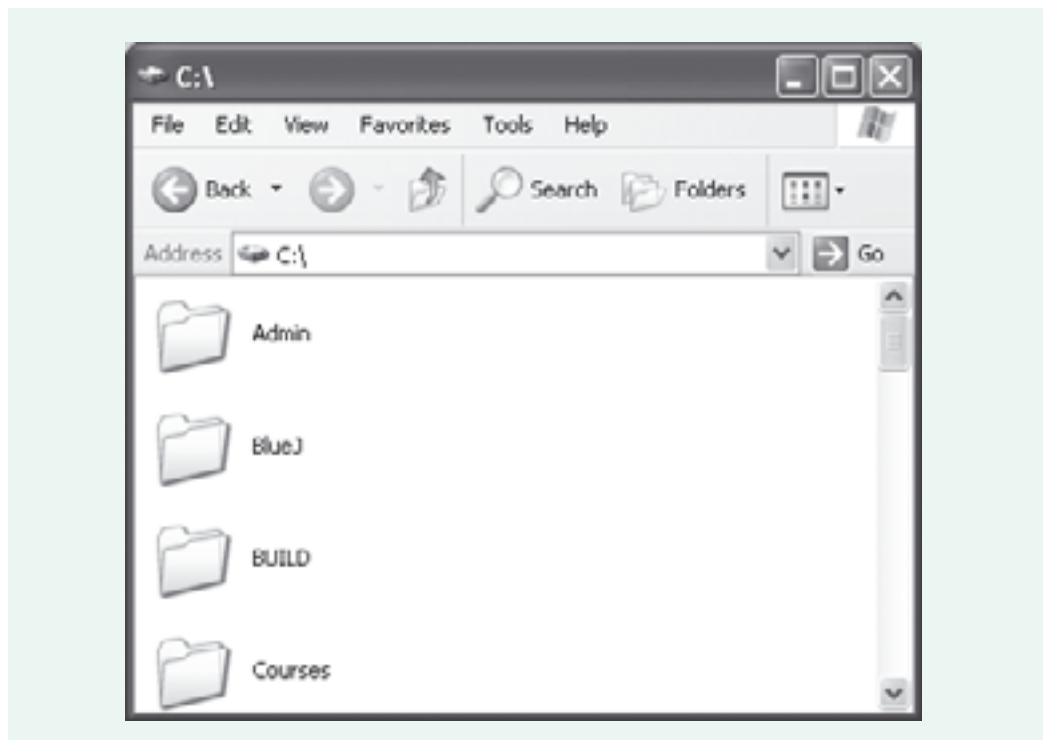


Figure 21 A window onto folders

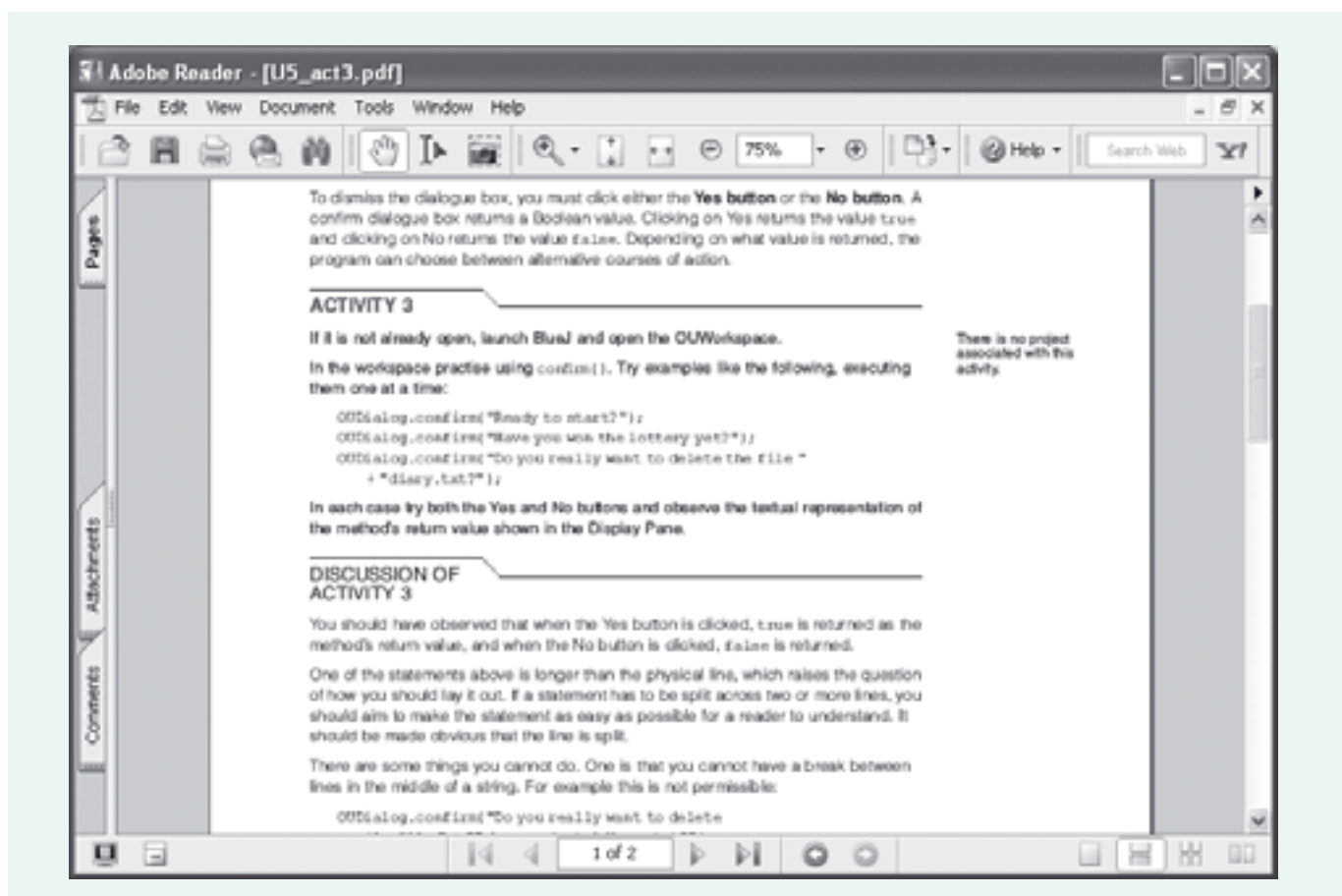


Figure 22 A window onto a document

Exercise 11

In what ways is a user interface window inconsistent with a real window?

Discussion.....

User interface windows have many characteristics not shared by real windows. For example, their size and shape can be altered, and in many GUI interfaces they can be reduced to an icon. Also, more subtly, when an interface window is moved, the objects you 'see' in it move too.

Reducing to an icon means that the window disappears and is represented by an icon of some sort.

As Exercise 11 shows, windows are an inconsistent metaphor. In fact, the metaphor is really rather slight: although it is called a window, users do not expect it to behave like a real-world window. It is regarded as more like a *container* which can be opened, closed and moved about. Nevertheless, windows are an extremely successful device, particularly as they allow for direct manipulation. For example, copying between folders is merely a matter of dragging icons between windows.

A **dialogue box** is a particular type of window which gives the user information. It may also allow the user to enter information and to select from a set of options specified by buttons. Figure 23 illustrates a dialogue box.

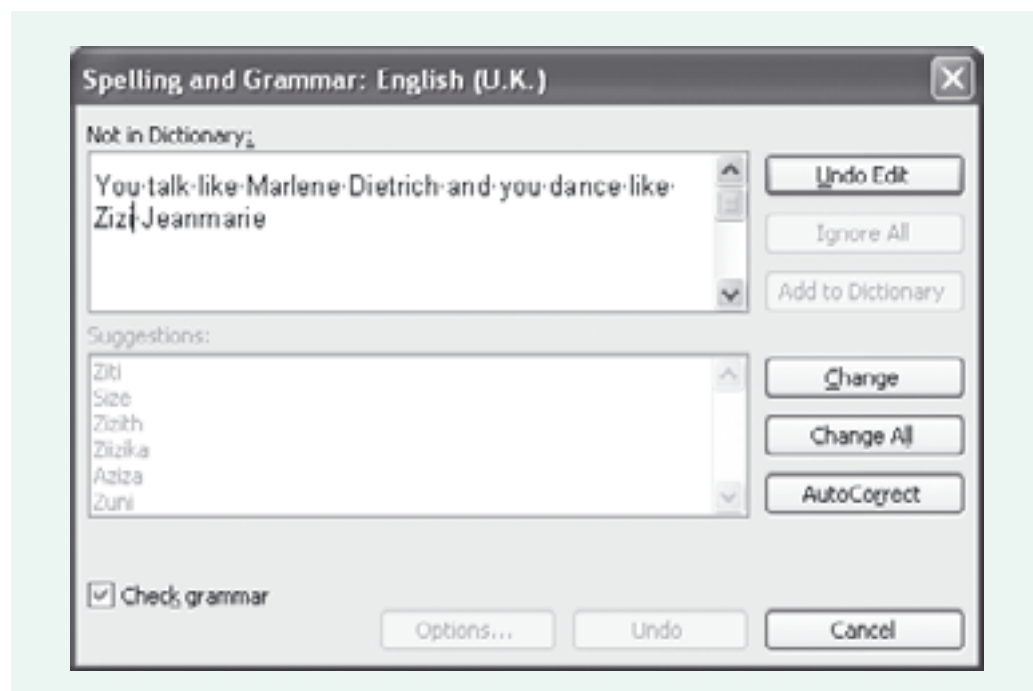


Figure 23 A dialogue box

Dialogue boxes have different characteristics from normal windows. For example a dialogue box can be used to demand an immediate response from the user. In most GUI interfaces, they cannot be resized, and in some they cannot even be moved.

5.5 Other widgets

There are many types of widget in common use within interfaces, far too many for us to itemise here. Here are some examples, with descriptions of their commonly associated functionality.

- ▶ A **list** allows the user to select one (or, rarely, more) pieces of information to provide to the system. The dialogue box in Figure 23 above contains a list, labelled **Suggestions:**. A list may or may not have a scroll bar.
- ▶ A **list box** is a combination of a text box (see below) and a list: the user can select from a list (which may drop down), or type into the box. An example is the **Font** list box in Figure 20.
- ▶ **Text fields** (or **text boxes**) are areas into which the user can type, or which can be used to display information, or both. A text field accommodates just a single line of text.
- ▶ **Text areas** are just like text fields, but they can accommodate multiple lines of text. They may or may not have scroll bars. Like text boxes, text areas can be set by the programmer to be input only, output only, or for both input and output. The dialogue box in Figure 23 contains a text area (with scroll bar), labelled **Not in Dictionary:**.
- ▶ **Tabbed panes** are separate areas of an interface which appear to be stacked on top of one another; a particular tabbed pane is accessed by clicking on a tab at the top. Figure 20 displays the tabbed pane called **Font**; 'behind' it are two other tabbed panes, of which only the tab are currently visible, and which are named **Character Spacing** and **Text Effects**.
- ▶ **Labels** usually give textual information about other widgets. Usually this information is unchanging, but it may be dynamic, as mentioned below.
- ▶ A **slider** is similar in appearance to a scroll bar and acts as a device to enter a number or other value, by moving the slider bar.

The range of the values, and the smallest allowable increment, depend on the particular application. Sliders are often used in conjunction with text boxes, which enable the user to type a value and/or display the currently chosen value; however, Figure 24 shows a slider being used in conjunction with a dynamic label whose text (here, 1024 by 768 pixels) changes to display the currently chosen value. Sliders are also particularly useful for entering continuous data, such as wavelengths, allowing the input data to be varied smoothly and quickly until the desired level is reached.

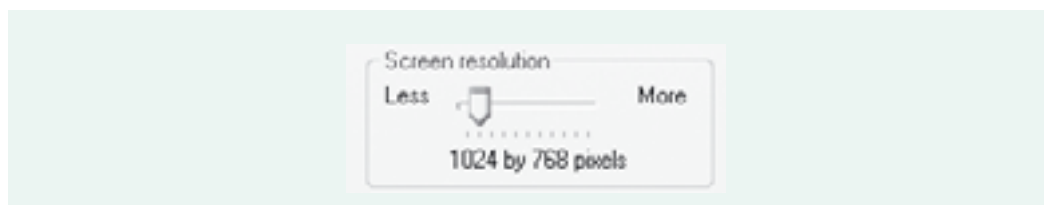


Figure 24 A slider

Exercise 12

The dialogue box shown in Figure 25 appears when the user opts to print a document. It involves a number of the GUI widgets discussed above. Label each of the main widgets.

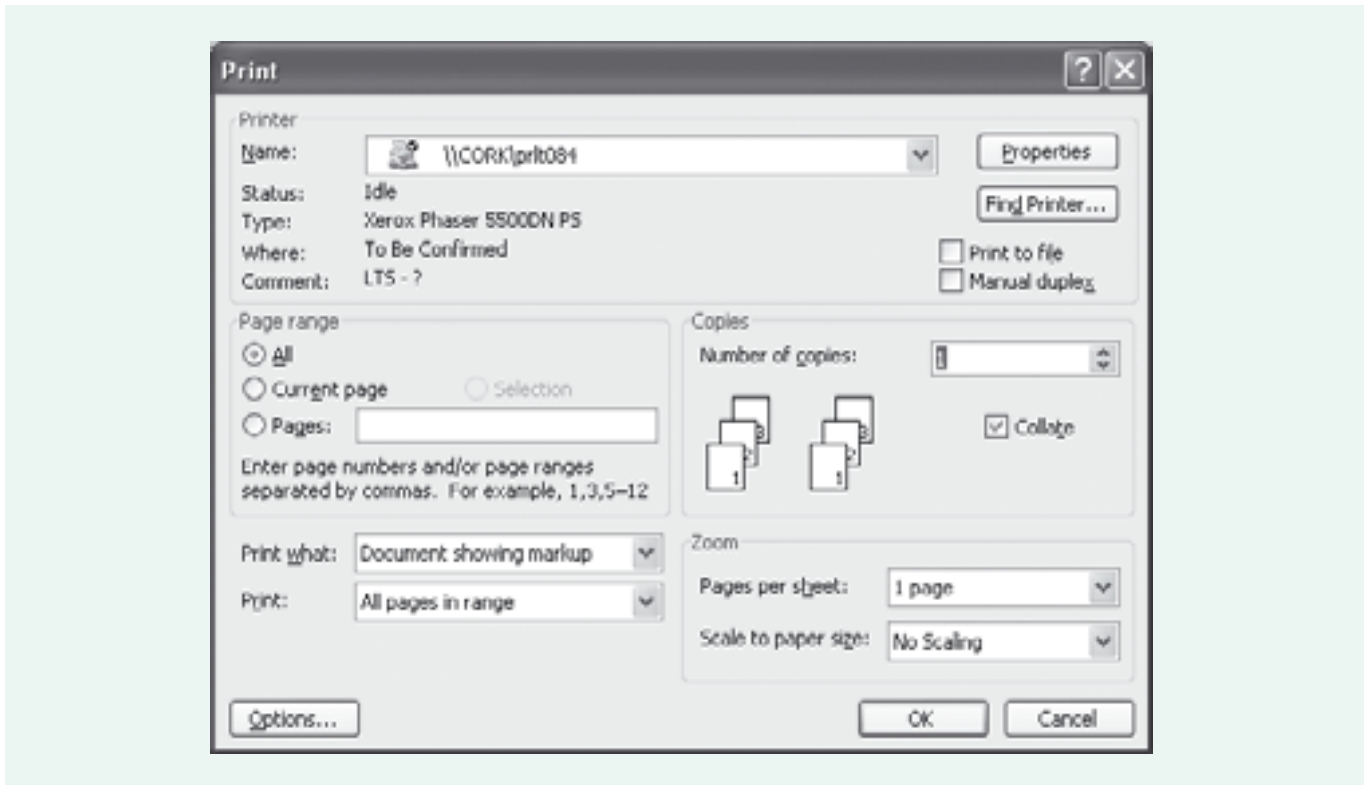


Figure 25 A print dialogue box

Discussion.....

Figure 26 shows the dialogue box with the main widgets labelled.

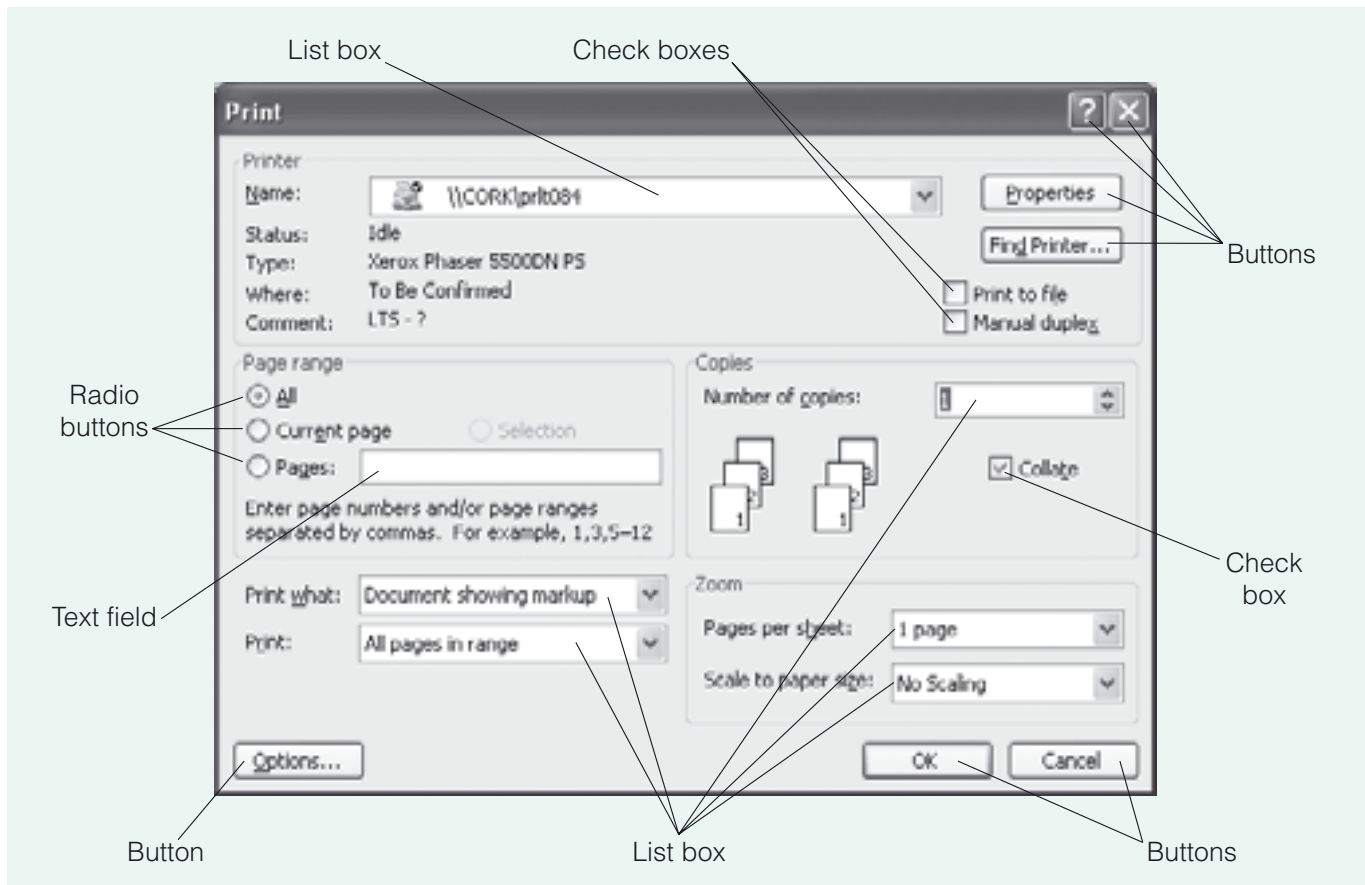


Figure 26 Labelled print dialogue box

The next exercise asks you to draw on what you have learnt about widget functionality in this section, as well as what you know about widgets from your experience as a computer user.

Exercise 13

By considering the functionality required, suggest what kind of widget (or combination of widgets) might be appropriate for each of the following purposes. Give reasons for your answers.

- The user is to indicate their country of birth, out of over 100 possible countries.
- The user is to select tourist attractions in a city, out of 20 available, that they might like to visit, as part of planning for a holiday.
- There has been a power cut, and the system is suddenly running on emergency power. The user is asked if they wish to save their work before emergency power runs out.
- The user is to give their family name. This is one of 6 similar items of information that the user may provide in any order.
- The system is to display the text of a chapter of a book.
- The user is to set any number of several characteristics of a piece of text, such as italic, bold and underline.

Discussion.....

Bear in mind that the widgets suggested below are just likely possibilities. The actual choice of widgets would depend not only on functionality, but on other design concepts, the rest of the interface and, of course, the user.

- (a) A list box, or a list with a scroll bar, allowing a single choice, might be suitable. Radio buttons would enable the same functionality, but are not an obvious candidate because with so many options the screen would be very cluttered (though this is not essentially a functionality concern!).
- (b) Check boxes could be appropriate. If, as in (a), the number of options precludes this, then a list allowing multiple choices might be used.
- (c) A dialogue box is appropriate, because it could force the user to respond immediately.
- (d) Just one line of free input is required, making a text field a good fit. Since it is desirable to allow the input to be provided in any order, six text fields are more suitable than a series of dialogue boxes, which would force the user to attend to one particular item at a time.
- (e) A window with a scroll bar would be appropriate, allowing the display of any amount of text. Unless it is a very short chapter, a text area is likely to require considerable scrolling and hence be less appropriate.
- (f) Check boxes would be appropriate, and are perhaps the most obvious choice, though a list allowing multiple choices could also be used.

In fact, in some word processors this is actually achieved through what may initially look like buttons. Examples are the toolbar button-like widgets labelled with **B**, *I* and U in Figure 27. In reality, 'buttons' like these function like check boxes, toggling between two different states. When clicked on, they 'depress' to show that a style has been selected (equivalent to a tick in a check box); and when clicked on again, they return to their original form and deselect the style (equivalent to an empty check box).

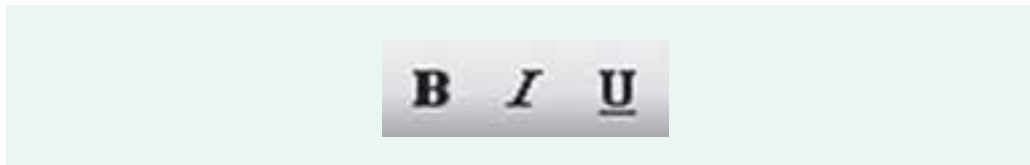


Figure 27 Changing styles in a word processor

In this section we have given a brief overview of the functionality of some widgets in common use. Of course, there are many more widgets than these, some of which you are undoubtedly familiar with. This abundance of widgets means that there are often many widgets, or combinations of widgets, to choose from as the discussion for Exercise 13 indicates.

Designers making such choices require substantial understanding of the potential and limitations of widgets, and of how they may effectively be combined. An introduction to HCI such as in M256 cannot provide a comprehensive survey, but your study of this section should give you a firm basis on which to analyse the functionality and design of widgets.

6

Summary

This unit began by introducing you to the notion of HCI, both as a science and as an area of practical activity, explaining some of the reasons why HCI in both its aspects is so important.

You were then introduced to some fundamental notions in HCI, such as the overriding importance of the user, and the concepts of design principles (we concentrated on good visibility, affordance and feedback) and interface metaphors (in particular, direct manipulation). You put these ideas into practice when evaluating example widgets and interfaces, as well as reviewing the functionality appropriate to certain widgets.

In the next unit, *Unit 12*, you will build on these ideas, putting them into practice as you:

- ▶ learn more about how the characteristics and needs of the user can be taken into account during user interface design.
- ▶ participate in designing a user interface from scratch;

You will also learn more about the connection between HCI and object-oriented ideas.

LEARNING OUTCOMES

Having studied this unit you should be able to:

- ▶ describe and use each of this unit's key terms (summarised in the Glossary);
- ▶ outline the importance of HCI, giving examples;
- ▶ outline what is involved in user-centred design;
- ▶ analyse user interfaces and widgets in terms of visibility, affordance and feedback;
- ▶ analyse user interface metaphors (including direct manipulation);
- ▶ select widgets appropriate for a particular task, and justify your selection.

Glossary

action button See button.

affordance The manipulations that something (e.g. a widget) appears to allow; that is, the actions that it appears should be carried out by the user to manipulate that thing.

button A widget typically used to enable the user to initiate a single action.

check box One of a set of widgets (there may be only one) enabling choice (including none) from a set of options which are not mutually exclusive.

command button See button.

command-line interface A user interface which requires the user to enter the names of commands and the names of the files (or other items of interest) to be manipulated.

design guideline A piece of guidance for the design of user interfaces, which typically is detailed and relevant to only specific kinds of system or specific contexts.

design principle A piece of guidance for the design of user interfaces, which typically is more abstract than a design guideline, and which should apply more or less universally to any kind of system in any context.

dialogue box A particular type of window which gives the user information, and may also allow the user to enter information and to select from a set of options.

direct manipulation A style of interaction based on the metaphor of physical manipulation.

extraordinary HCI Human–computer interaction in situations where the user has special needs.

feedback Information about the results of the user's action.

functionality (of a widget) The types of tasks the widget enables a user to perform, and the types of communication between user and system it mediates.

graphical user interface (GUI or GUI interface) A user interface that typically combines elements such as windows, buttons and menus: visual elements which can be manipulated by the user to interact with the system.

GUI or GUI interface See graphical user interface.

human–computer interaction (HCI) The study of how computers and people work together. The term HCI is used to refer both to the scientific study of how humans and computers interact, and (the focus in this course) to the practical business of designing user interfaces to enable such interactions.

icon A small representational picture used in a GUI; for example, to label a button.

interaction design The design of any aspects of a system that affect the quality of a user's interaction.

interactive product Any kind of computer-based device that requires users to engage with it.

label Information, usually textual, about a widget.

learnt affordance A view of affordance which stresses that the way a user perceives how something should be manipulated is not a property of the thing alone, but a property of the combination of the thing and the user's background, skill, capabilities and training.

list A widget that allows the user to select one (or, rarely, more) relevant pieces of information to provide to the system.

list box A widget which combines a text box and a list.

mental model Specific existing knowledge, often of the physical world, which users are typically expected to possess, and upon which a user interface metaphor is based.

menu A widget which groups together several items from which the user may select. Pull-down, pop-up and contextual menus are examples of different types of menu.

menu bar A widget that takes the form of a bar, appearing typically along the top of an application window, and which provides a way of hierarchically grouping commands. The words along the menu bar represent major groupings of commands, arranged in menus.

menu interface A user interface which presents the user with a sequence of sets of options from each of which the user selects, typically by pressing a button or typing in a number.

radio button One of a set of widgets for enabling choice between mutually exclusive options, precisely one of which must be selected.

safety-critical system A system operating in an environment in which safety is particularly crucial.

screen See window.

slider A widget that acts as a device to input or output a number or other value, by moving a bar.

socio-technical system A system comprising both human and technical elements.

tabbed pane One of several separate areas of an interface which appear to be stacked on top of one another, and which are individually brought to the foreground by clicking on a tab at the top.

text area A widget which is like a text field but which can accommodate multiple lines of text.

text box See text field.

text field A widget providing an area into which the user can type, or which can be used to display information, or both. A text field accommodates just a single line of text.

usability Refers to how easily and quickly a product enables users to achieve their goals, how quickly they learn to use the product's interface and their attitude to the interface.

user interface The features of a system the user can use to communicate with or control the system, and those which the system can use to communicate with, or influence the user. A user interface has three aspects: hardware; visual elements; and interaction design.

user needs analysis The process of finding out as much as possible about the people who will be using the system.

user-centred design A design culture or design process where users' needs, wants, expectations and limitations are given serious, detailed attention at all stages.

user interface metaphor A metaphor applied in the design of a user interface.

virtual reality system A system which gives the user a sense of three-dimensional direct experience or physical presence via cues that may be visual, aural or haptic, and which uses computer-generated 'objects' that are manipulated like their real-world equivalents by picking up, turning around, throwing etc.

visibility The extent to which the purpose of something (e.g. a widget) is apparent, i.e. how clear it is what the thing is for.

widget A visual element in a GUI such as a button, a list etc.

window A window- or container-like widget, typically allowing a view onto some aspect of a system or application.

References

Norman, D. A. (2002) *The Design of Everyday Things*, Basic Books Inc., New York, USA.

Acknowledgements

Grateful acknowledgement is made to the following sources.

Figure 1: www.altavista.com

Figure 2: www.google.com

Figure 3: www.google.com

Figure 4: Brooks Jr, F. P., Ouh-Young, M., Batter, J. J. and Kilpatrick, P. J, (1990) 'Project GROPE – Haptic displays for scientific visualisation', *ACM Computer Graphics*, vol.24, no.4, pp. 177–86.

Figure 5: Edwards, D. N. (1991) *Speech Synthesis: Technology for Disabled People*, Sage Publications Ltd.

Index

A

action button 38
affordance 23
Apple Macintosh 13, 18
aural cue 12

B

button 38

C

check box 40
cognitive psychology 19
command button 38
command-line interface 18
core system 5, 32
cue
 aural 12
 haptic 12
 visual 12

D

design guideline 21
design principle 21
desktop metaphor 30
dialogue box 43
direct manipulation 32

E

Equaliser system 14
extraordinary HCI 14

F

feedback 24
functionality (of a widget) 36

G

graphical user interface (GUI/GUI
 interface) 6, 18
GROPE (force-feedback
 interface) 12

H

haptic cue 12
Hawking, Stephen 14
human–computer interaction
 (HCI) 5

I

interaction design 17
interactive product 6
iPod 11

J

Java 18

L

label 44
learnt affordance 26
linguistics 19
list 44
list box 44

M

Macintosh 18
mental model 29
menu 36
 bar 36
 interface 19
 pull-down 36
metaphor
 desktop 30
 direct manipulation 32
 user interface 28

MP3 player 11, 32

P

pull-down menu 36

R

radio button 40

S

safety-critical system 8
screen 41
slider 44
Smalltalk 18
socio-technical system 16, 19
sociology 19

T

tabbed pane 44
text area 44
text box 44
text field 44

U

usability 8
user interface 8, 16
 of iPods 11
 of search engines 9
 related disaster 9
user interface metaphor 28
user needs analysis 20
user-centred design 19

V

virtual reality systems 12
visibility 22
visual cue 12

W

widget 17
window 41
World Wide Web 12

X

Xerox PARC 18

